

Real-Time FPGA Electromagnetic Field Visualiser

Analog Team

FDTD Team

Rendering Team

June 2026

Contents

1	The Problem: Visualising the Invisible	3
1.1	Project specifications and requirements	3
2	Overall Architecture and Project Management	4
2.1	Project Management	4
2.1.1	Team structure	4
2.1.2	Working practices	4
2.1.3	Deliverable structure	5
2.1.4	Timeline	5
3	The FDTD Solver and Physics	6
3.1	Maxwell's Equations	6
3.1.1	Meaning of each symbol	6
3.1.2	Why we set $\mathbf{J} = \mathbf{0}$	6
3.1.3	Dynamic case: the wave equation ($\partial/\partial t \neq 0$)	7
3.1.4	Which equations are actively solved in the FDTD engine	7
3.2	Field Component Reduction: From Six to Three	7
3.2.1	Step 1: The full 3D coupling	7
3.2.2	Step 2: Applying translational symmetry in z	8
3.2.3	Step 3: The two decoupled field systems	9
3.3	Why Two Dimensions	10
3.4	Discretisation	10
3.5	Choosing the Algorithm	12
3.6	The Coefficients and Their Limits	13
3.7	Number Format	13
3.8	Pipelining the Update	13
3.9	Boundaries and the Absorbing Layer	14
3.10	From Three Passes to Two	15
3.11	Lane Parallelism and the Halo	16
3.12	Performance Comparison	18
3.13	Verification	18
4	The Renderer	20
4.1	Foundations and the baseline design	20
4.1.1	The rendering problem	20
4.1.2	The fixed pixel pipeline	20
4.1.3	Number representation	21
4.1.4	Choosing the march depth	21
4.1.5	The baseline architecture	22
4.2	Shading interpolation: the free visual win	24
4.2.1	Reusing the reads the normal already makes	24

4.2.2	The interpolation mathematics	24
4.2.3	Cost	25
4.3	Folding and the FIFO scanout	26
4.3.1	Clocking and the two domains	26
4.3.2	Matching throughput and latency to the display	26
4.3.3	Reliability	27
4.3.4	The limit of the FIFO scanout	27
4.4	Silhouette interpolation and the folded board design	28
4.4.1	Design 3 : bilinear in the marcher, and the cost explosion	28
4.4.2	Folding the reads : Design 4	28
4.4.3	Folding the multipliers: from 70% to 26% DSP	28
4.5	Result :	30
4.5.1	Scaling to 128x128 is still not possible without parallel lanes	30
4.6	The frame buffer and parallel lanes	31
4.6.1	Decoupling with a frame buffer	31
4.6.2	Recovering speed with parallel lanes	32
4.6.3	Why this uses fewer resources	33
4.7	Final comparison: five candidate designs	34
4.7.1	Fitting the DSP budget	34
4.7.2	The final design	35
5	Interface and System Integration	36
5.1	System Overview	36
5.2	Building the Design from a Script	36
5.3	Clocking	37
5.4	Generating the Source	37
5.5	The Moving Source and the Doppler Effect	38
5.6	Decoupling the Solver and the Display	39
5.7	The Bridge to the Renderer	39
5.8	Driving It from the Processor	40
5.9	Multi-Lane Parallelism	40
5.10	The Two Demonstrations	41
6	User Interface, Hardware and Demonstrations	44
6.1	What the demonstrator does, and the physics behind it	44
6.2	The user interface	46
6.3	Power supply and analog conditioning	46
6.4	The microcontroller and the control link	47
6.5	The touch panel: sensing and calibration	47
6.6	Enclosure and mechanical design (box modelling)	49
6.7	Interactive Wave Simulation	50
6.7.1	Panel 1: Electric field $ \mathbf{E} (x, y, t)$	50
6.7.2	Panel 2: Magnetic field $B_z(x, y, t)$	51
6.7.3	Panel 3: Poynting vector $ \mathbf{S} (x, y, t)$	51
6.7.4	Relationship between the three panels	51
6.7.5	More demonstrations	52
7	Evaluation and Conclusion	53
7.1	Integrated system performance	53
7.2	Requirements met	53
7.3	Limitations and future work	53
7.4	Conclusion	53

Chapter 1

The Problem: Visualising the Invisible

Abstract. This report details the physics underlying a real-time, FPGA-accelerated electromagnetic-field visualiser designed as an educational tool for electrical engineering students. The system makes Maxwell's equations physically tangible by solving the time-dependent field equations numerically and rendering the resulting fields as interactive 3D surfaces. Although Maxwell's equations can be solved analytically in simple cases, the fields themselves are invisible and difficult to interpret intuitively. The visualiser addresses this problem by allowing students to place wave sources, add reflecting boundaries, vary material properties, and observe propagation, interference, diffraction, and energy flow in real time.

1.1 Project specifications and requirements

Chapter 2

Overall Architecture and Project Management

2.1 Project Management

2.1.1 Team structure

The project was divided into three sub-teams, each responsible for a distinct subsystem as shown in Figure 2.1.

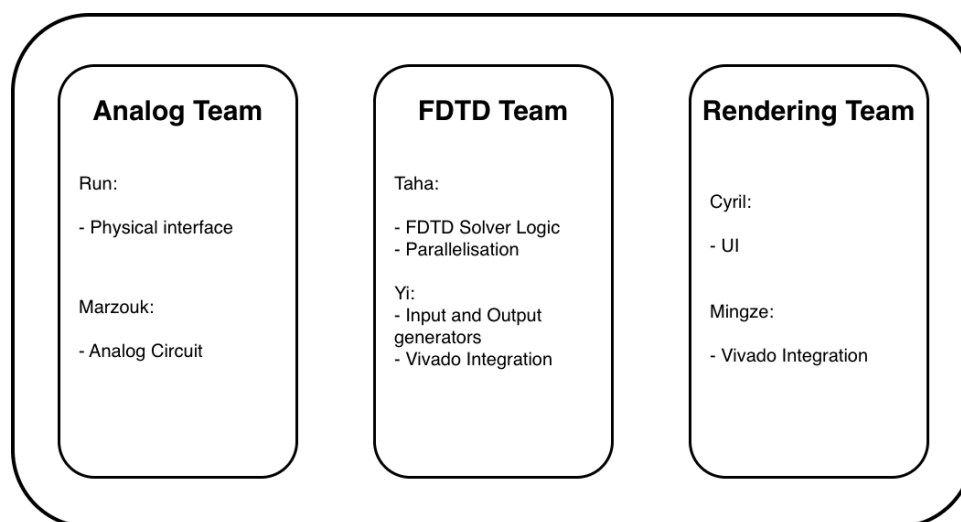


Figure 2.1: Division of work across the three sub-teams.

2.1.2 Working practices

Daily co-located work. The full group of six worked together in the same space throughout the project. Questions between sub-teams were answered in minutes and debugging happened collaboratively across the EEE/EIE split, rather than asynchronously over message threads.

Frequent structured syncs. A one-hour planning meeting was held every two days. Each meeting reviewed what had been completed, identified blockers, and replanned the remaining timeline. This kept all three sub-teams aligned and prevented any subsystem from falling behind without the rest of the group being aware.

Interfaces agreed upfront. The data interfaces between subsystems were defined at the start of the project before implementation began. The FDTD team committed to writing field data to specific BRAM addresses in a defined format; the rendering team committed to reading from those addresses. The user-interface team committed to delivering touch coordinates, switch

states, and knob values in a fixed control packet at a defined rate. This allowed all three sub-teams to build in parallel and integrate cleanly without waiting for each other.

Disciplined version control. The codebase was managed on Git with one branch per sub-team. The main branch was kept always-buildable. Merges were conducted via peer-reviewed pull requests, so parallel development across three sub-teams never broke the working skeleton.

2.1.3 Deliverable structure

Each sub-team worked toward four internal deliverables, providing intermediate checkpoints against the project timeline as shown in Figure 2.2.

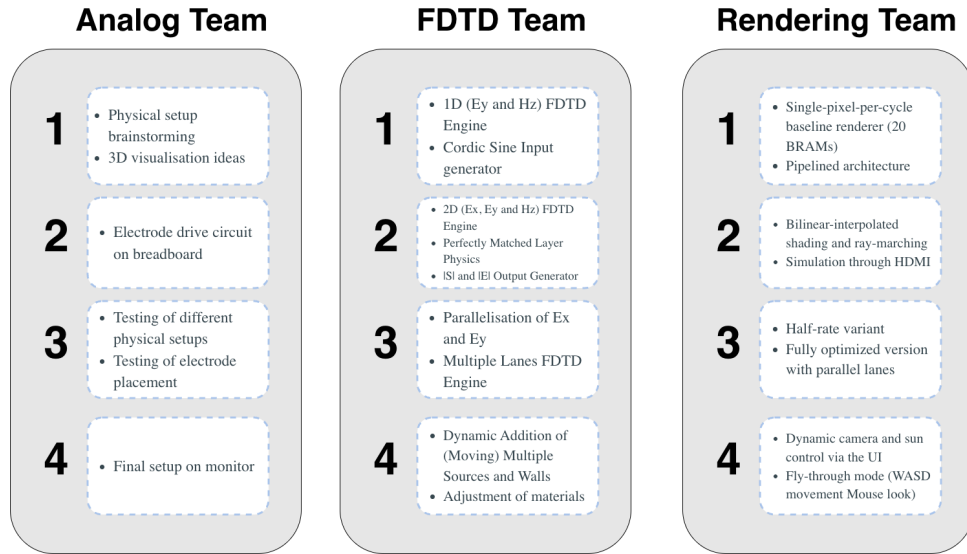


Figure 2.2: Deliverable / MVP roadmap across the three sub-teams.

2.1.4 Timeline

The timeline to complete these deliverables is shown in Figure 2.3.

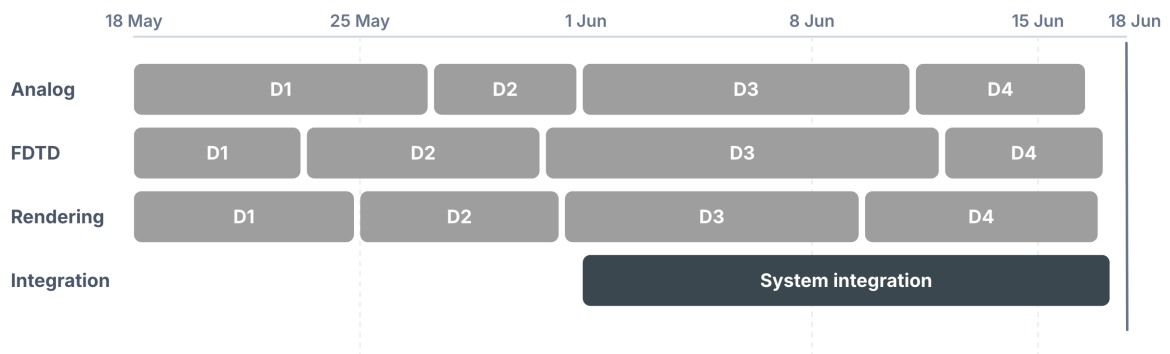


Figure 2.3: Project timeline across the three sub-teams.

Chapter 3

The FDTD Solver and Physics

3.1 Maxwell's Equations

Our entire system relies on actively solving Maxwell's equations. In differential form:

$$\nabla \cdot \mathbf{E} = \frac{\rho}{\varepsilon_0} \quad (\text{Gauss – electric})$$

$$\nabla \cdot \mathbf{B} = 0 \quad (\text{Gauss – magnetic})$$

$$\nabla \times \mathbf{E} = -\frac{\partial \mathbf{B}}{\partial t} \quad (\text{Faraday})$$

$$\nabla \times \mathbf{B} = \mu_0 \mathbf{J} + \mu_0 \varepsilon_0 \frac{\partial \mathbf{E}}{\partial t} \quad (\text{Ampère–Maxwell})$$

3.1.1 Meaning of each symbol

The symbols in the equations above are defined as follows.

$\mathbf{E}$ (V m^{-1}) is the electric field vector.

$\mathbf{B}$ (T) is the magnetic flux density.

ρ (C m^{-3}) is the free electric charge density. $\mathbf{J}$ (A m^{-2}) is the current density. It is the amount of charge that passes through an elementary plate in the unit of time and in the direction perpendicular to the plate.

$\varepsilon_0 = 8.85 \times 10^{-12} \text{ F m}^{-1}$ is the permittivity of free space.

$\mu_0 = 4\pi \times 10^{-7} \text{ H m}^{-1}$ is the permeability of free space.

$$c = \frac{1}{\sqrt{\mu_0 \varepsilon_0}} \approx 3 \times 10^8 \text{ m s}^{-1} \quad (3.1)$$

The divergence $\nabla \cdot$ shows if the field is converging or diverging and by how much. Positive divergence means field lines originate there, negative means they terminate there.

The curl $\nabla \times$ has a magnitude which shows the amount of rotation of the field at a particular point. The sign shows the direction of rotation.

3.1.2 Why we set $\mathbf{J} = 0$

Setting $\mathbf{J} = 0$ means we are simulating a region with no free currents flowing through it. This makes sense because the simulation domain is free space or a dielectric medium, so there are no wires, conductors, and charged particles moving through the field.

With $\mathbf{J} = 0$ everywhere, the Ampère–Maxwell equation simplifies to:

$$\nabla \times \mathbf{B} = \mu_0 \varepsilon_0 \frac{\partial \mathbf{E}}{\partial t} \quad (3.2)$$

Similarly, in a charge-free medium ($\rho = 0$), Gauss's law for electricity becomes simply $\nabla \cdot \mathbf{E} = 0$.

3.1.3 Dynamic case: the wave equation ($\partial/\partial t \neq 0$)

When fields vary in time, Faraday's equation and the simplified Ampère–Maxwell equation (3.2) are coupled. Taking the curl of the Faraday equation:

$$\nabla \times (\nabla \times \mathbf{E}) = -\frac{\partial}{\partial t} = -\mu_0 \varepsilon_0 \frac{\partial^2 \mathbf{E}}{\partial t^2} \quad (3.3)$$

Applying the vector identity $\nabla \times (\nabla \times \mathbf{E}) = \nabla(\nabla \cdot \mathbf{E}) - \nabla^2 \mathbf{E}$ and using $\nabla \cdot \mathbf{E} = 0$ in a charge-free region:

$$\boxed{\nabla^2 \mathbf{E} = \mu_0 \varepsilon_0 \frac{\partial^2 \mathbf{E}}{\partial t^2}} \quad (3.4)$$

This is the electromagnetic wave equation. The wave speed in free space is $c = 1/\sqrt{\mu_0 \varepsilon_0}$ as given in (3.1). In a medium with relative permittivity ε_r and relative permeability μ_r :

$$v = \frac{c}{\sqrt{\varepsilon_r \mu_r}} \quad (3.5)$$

3.1.4 Which equations are actively solved in the FDTD engine

The four Maxwell equations do not all require explicit solution in the FDTD loop. The two divergence equations, Gauss's law (Gauss – electric) and the magnetic Gauss law (Gauss – magnetic), are automatically satisfied for all time as a consequence of the curl equations, as the fields start at 0 everywhere.

The FDTD engine therefore explicitly solves only two equations:

- The Faraday equation, which updates B_z from the curl of $\mathbf{E}$.
- The simplified Ampère–Maxwell equation (3.2), which updates E_x and E_y from the gradient of B_z .

3.2 Field Component Reduction: From Six to Three

3.2.1 Step 1: The full 3D coupling

In full three-dimensional space, Maxwell's curl equations couple all six field components together. Writing the Faraday equation $\nabla \times \mathbf{E} = -\partial \mathbf{B}/\partial t$ and the Ampère–Maxwell equation $\nabla \times \mathbf{B} = \mu_0 \varepsilon_0 \partial \mathbf{E}/\partial t$ in component form gives six equations.

From Faraday's law:

$$\frac{\partial B_x}{\partial t} = \frac{\partial E_y}{\partial z} - \frac{\partial E_z}{\partial y} \quad (3.6)$$

$$\frac{\partial B_y}{\partial t} = \frac{\partial E_z}{\partial x} - \frac{\partial E_x}{\partial z} \quad (3.7)$$

$$\frac{\partial B_z}{\partial t} = \frac{\partial E_x}{\partial y} - \frac{\partial E_y}{\partial x} \quad (3.8)$$

From Ampère–Maxwell:

$$\frac{\partial E_x}{\partial t} = \frac{1}{\mu_0 \varepsilon_0} \left(\frac{\partial B_z}{\partial y} - \frac{\partial B_y}{\partial z} \right) \quad (3.9)$$

$$\frac{\partial E_y}{\partial t} = \frac{1}{\mu_0 \varepsilon_0} \left(\frac{\partial B_x}{\partial z} - \frac{\partial B_z}{\partial x} \right) \quad (3.10)$$

$$\frac{\partial E_z}{\partial t} = \frac{1}{\mu_0 \varepsilon_0} \left(\frac{\partial B_y}{\partial x} - \frac{\partial B_x}{\partial y} \right) \quad (3.11)$$

To update any single component requires two others

3.2.2 Step 2: Applying translational symmetry in z

Our simulation domain is two-dimensional and the model is assumed to be uniform in the out-of-plane z direction, so every z -partial derivative is identically zero:

$$\frac{\partial}{\partial z} = 0 \quad (3.12)$$

Faraday equations after $\partial/\partial z = 0$:

$$\frac{\partial B_x}{\partial t} = \frac{\cancel{\partial E_y}}{\cancel{\partial z}} - \frac{\partial E_z}{\partial y} \longrightarrow \frac{\partial B_x}{\partial t} = -\frac{\partial E_z}{\partial y} \quad (3.13)$$

$$\frac{\partial B_y}{\partial t} = \frac{\partial E_z}{\partial x} - \frac{\cancel{\partial E_x}}{\cancel{\partial z}} \longrightarrow \frac{\partial B_y}{\partial t} = +\frac{\partial E_z}{\partial x} \quad (3.14)$$

$$\frac{\partial B_z}{\partial t} = \frac{\partial E_x}{\partial y} - \frac{\partial E_y}{\partial x} \longrightarrow \frac{\partial B_z}{\partial t} = \frac{\partial E_x}{\partial y} - \frac{\partial E_y}{\partial x} \quad (\text{unchanged}) \quad (3.15)$$

Ampère–Maxwell equations after $\partial/\partial z = 0$:

$$\frac{\partial E_x}{\partial t} = \frac{1}{\mu_0 \varepsilon_0} \left(\frac{\partial B_z}{\partial y} - \frac{\cancel{\partial B_y}}{\cancel{\partial z}} \right) \longrightarrow \frac{\partial E_x}{\partial t} = \frac{1}{\mu_0 \varepsilon_0} \frac{\partial B_z}{\partial y} \quad (3.16)$$

$$\frac{\partial E_y}{\partial t} = \frac{1}{\mu_0 \varepsilon_0} \left(\frac{\cancel{\partial B_x}}{\cancel{\partial z}} - \frac{\partial B_z}{\partial x} \right) \longrightarrow \frac{\partial E_y}{\partial t} = -\frac{1}{\mu_0 \varepsilon_0} \frac{\partial B_z}{\partial x} \quad (3.17)$$

$$\frac{\partial E_z}{\partial t} = \frac{1}{\mu_0 \varepsilon_0} \left(\frac{\partial B_y}{\partial x} - \frac{\partial B_x}{\partial y} \right) \longrightarrow \frac{\partial E_z}{\partial t} = \frac{1}{\mu_0 \varepsilon_0} \left(\frac{\partial B_y}{\partial x} - \frac{\partial B_x}{\partial y} \right) \quad (\text{unchanged}) \quad (3.18)$$

Inspecting the right-hand sides of the reduced equations shows that the two sets of variables are unrelated

No variable from one group appears on the right-hand side of any equation from the other group. The system has split into two completely independent sets of three equations. An example of the components of the wave can be seen in the figure below:

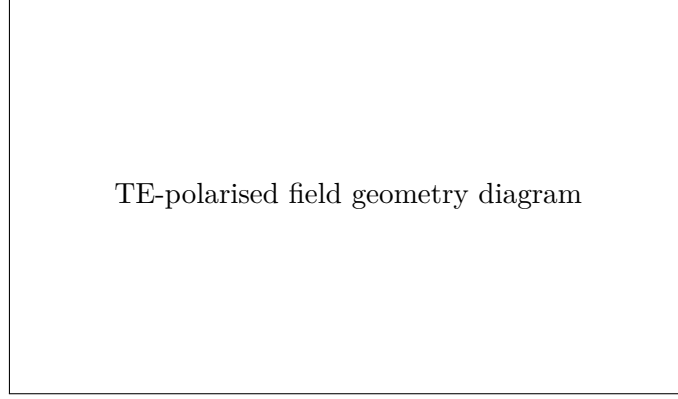


Figure 3.1: TE-polarised propagation between two parallel boundaries. This has the same field-component geometry as the TE system (E_x, E_y, B_z) used in our 2D FDTD simulation.

3.2.3 Step 3: The two decoupled field systems

The six equations separate exactly into two independent systems. This is an exact result, not an approximation.

TM system (E_z, B_x, B_y) :

$$\frac{\partial B_x}{\partial t} = -\frac{1}{\mu_0} \frac{\partial E_z}{\partial y} \quad (3.19)$$

$$\frac{\partial B_y}{\partial t} = +\frac{1}{\mu_0} \frac{\partial E_z}{\partial x} \quad (3.20)$$

$$\frac{\partial E_z}{\partial t} = \frac{1}{\varepsilon_0} \left(\frac{\partial B_y}{\partial x} - \frac{\partial B_x}{\partial y} \right) \quad (3.21)$$

TE system (E_x, E_y, B_z) :

$$\frac{\partial B_z}{\partial t} = -\frac{1}{\mu_0} \left(\frac{\partial E_y}{\partial x} - \frac{\partial E_x}{\partial y} \right) \quad (3.22)$$

$$\frac{\partial E_x}{\partial t} = +\frac{1}{\varepsilon_0} \frac{\partial B_z}{\partial y} \quad (3.23)$$

$$\frac{\partial E_y}{\partial t} = -\frac{1}{\varepsilon_0} \frac{\partial B_z}{\partial x} \quad (3.24)$$

E_z points perpendicular to the simulation plane. B_x and B_y lie within the plane. E_x and E_y lie within the simulation plane. B_z points perpendicular to the plane.

If the TE system is initialised to zero and only the TM system is excited, the TE components remain zero for all time, and vice versa. We simulate only the TE system, so the TM components do not need to be stored or computed.

The TE system, storing E_x , E_y , and $B_z = \mu_0 H_z$, was selected for three reasons:

1. **Poynting vector lies in-plane:** The electromagnetic power flux $\mathbf{S} = \mathbf{E} \times \mathbf{B}/\mu_0$ for the TE system has components:

$$S_x = \frac{E_y B_z}{\mu_0}, \quad S_y = -\frac{E_x B_z}{\mu_0}, \quad S_z = 0 \quad (3.25)$$

Both S_x and S_y lie within the xy plane and can be visualised directly. For the TM system, the in-plane energy-flow representation is less direct for this visualisation.

2. **Natural representation on a 2D grid:** The electric-field components E_x and E_y lie directly in the simulation plane, while B_z is perpendicular to it. This arrangement matches the Cartesian grid and allows field direction, magnitude, and energy flow to be computed without reconstructing missing in-plane components.
3. **Nicer visualisation:** Two electric field components give both magnitude $|\mathbf{E}| = \sqrt{E_x^2 + E_y^2}$ and direction $\theta = \arctan(E_y/E_x)$. the TM system provides only the scalar E_z , giving magnitude alone.

All electromagnetism material and information in this section, including equations, derivations, and the TE-polarisation diagram (Figure 3.1) are drawn from the second-year electromagnetism lecture notes.

3.3 Why Two Dimensions

The TE equations we previously chose that our FDTD algorithm is trying to solve are:

$$\begin{aligned}\epsilon \frac{\partial E_x}{\partial t} &= \frac{\partial H_z}{\partial y} \\ \epsilon \frac{\partial E_y}{\partial t} &= -\frac{\partial H_z}{\partial x} \\ \mu \frac{\partial H_z}{\partial t} &= \frac{\partial E_x}{\partial y} - \frac{\partial E_y}{\partial x}\end{aligned}$$

The reason for choosing 3 fields in 2 dimensions instead of 6 fields in 3 dimensions is because of our strict memory limitation of 4.9 Mbits of BRAM.

If we take the example of a 64 length field in 2 dimensions with 3 fields at 16 bits (Q3.13) we would need 6/140 BRAMs meanwhile computing 6 fields in 3 dimensions would require 768/140 BRAMS (5x more than we have available). These calculations don't account for memory needed for calculating height maps, double buffers and the significant memory demands of the renderer, nor accounting for the fact that our final length is 128. This shows that due to the memory constraints of the FPGA, even if we deployed creative use of DDR caching it is not feasible to compute FDTD in 3 dimensions of the Pynq Z1. DDR caching was determined not to be useful anyway as the rendering logic needs random access which would defeat the purpose of DDR caching.

3.4 Discretisation

Maxwell's Curl Equations deploy the use of PDEs which are extremely expensive to compute on the PL especially with the quantity of PDEs we want to compute. Therefore we chose to use the industry renowned FDTD method to simplify the calculations. It uses the following approximation which is 2nd order accurate. This approximation is obtained using the Taylor series expansion of the partial derivative

$$\frac{\partial H}{\partial x} \approx \frac{H(x + \frac{\Delta x}{2}) - H(x - \frac{\Delta x}{2})}{\Delta x}$$

Derivation of FDTD form for E_x :

$$\epsilon_0 \frac{\partial E_x}{\partial t} = \frac{\partial H_z}{\partial y} \quad (3.26)$$

Divide by ϵ_0 , substitute $H_z = B_z/\mu_0$, normalise $\epsilon_0\mu_0 = c^{-2} = 1$:

$$\frac{\partial E_x}{\partial t} = \frac{\partial B_z}{\partial y} \quad (3.27)$$

Taylor expand B_z about $[i, j]$ at $\pm \frac{\Delta y}{2}$ in the column (i) direction:

$$B_z \left[i + \frac{1}{2}, j \right] = B_z[i, j] + \frac{\Delta y}{2} \frac{\partial B_z}{\partial y} + \frac{1}{2} \left(\frac{\Delta y}{2} \right)^2 \frac{\partial^2 B_z}{\partial y^2} + O(\Delta y^3) \quad (3.28)$$

$$B_z \left[i - \frac{1}{2}, j \right] = B_z[i, j] - \frac{\Delta y}{2} \frac{\partial B_z}{\partial y} + \frac{1}{2} \left(\frac{\Delta y}{2} \right)^2 \frac{\partial^2 B_z}{\partial y^2} + O(\Delta y^3) \quad (3.29)$$

Subtract:

$$B_z \left[i + \frac{1}{2}, j \right] - B_z \left[i - \frac{1}{2}, j \right] = \Delta y \frac{\partial B_z}{\partial y} + O(\Delta y^3) \quad (3.30)$$

Dividing by Δy and rearrange:

$$\frac{\partial B_z}{\partial y} = \frac{B_z[i, j] - B_z[i - 1, j]}{\Delta y} + O(\Delta y^2) \quad (3.31)$$

Substituting into (2) and apply the same argument in time:

$$\frac{E_x^{n+1}[i, j] - E_x^n[i, j]}{\Delta t} = \frac{B_z^{n+\frac{1}{2}}[i, j] - B_z^{n+\frac{1}{2}}[i - 1, j]}{\Delta y} + O(\Delta t^2, \Delta y^2) \quad (3.32)$$

Rearrange:

$$E_x^{n+1}[i, j] = E_x^n[i, j] + \frac{\Delta t}{\Delta y} \left(B_z^{n+\frac{1}{2}}[i, j] - B_z^{n+\frac{1}{2}}[i - 1, j] \right) \quad (3.33)$$

Add PML coefficients c_a and c_b :

$$E_x^{n+1}[i, j] = c_a[i, j] \cdot E_x^n[i, j] - c_b \left(B_z^{n+\frac{1}{2}}[i, j] - B_z^{n+\frac{1}{2}}[i - 1, j] \right) \quad (3.34)$$

FDTD uses a Yee Cell Lattice to represent the magnetic and electric fields in 2D space with a half cell staggered approach, meaning the magnetic fields are at .5 indexes while the electric fields are at integer indexes. This means that Ey cells are updated by the change in its top and bottom Bz componenets, Ex are updated by the change in its left and right Bz components and Bz cells are updated by the change in the surrounding 2 Ex and 2 Ey components.

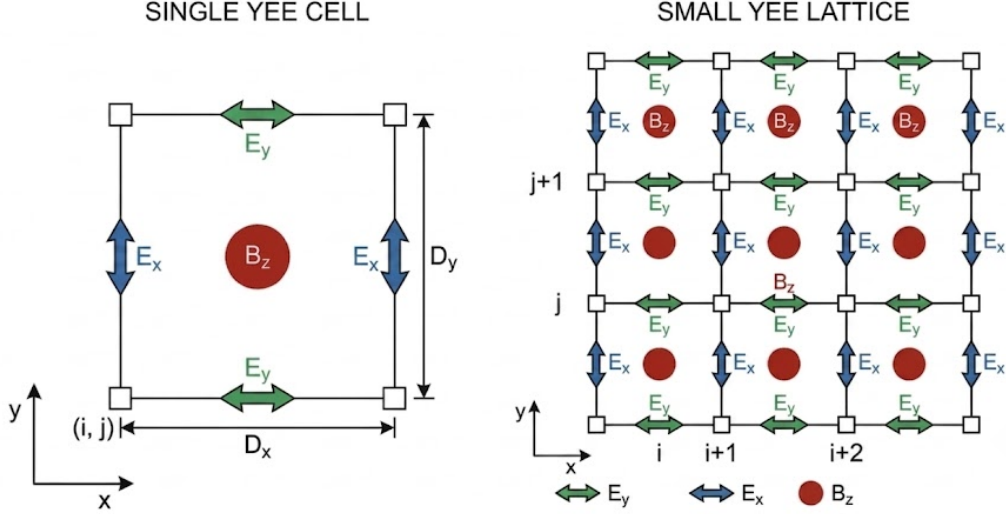


Figure 3.2: Yee cell lattice showing the half-cell staggering of the E_x , E_y and B_z field components

	$E_y[i][j]$	$E_x[i][j]$	$B_z[i][j]$	$B_z[i-1][j]$	$B_z[i][j-1]$	$E_y[i][j+1]$	$E_x[i+1][j]$
E_y update	×		×		×		
E_x update		×	×	×			
B_z update	×	×	×			×	×

Table 3.1: Read dependencies for each field update in the TE_z FDTD scheme (i = column, j = row).

This method relies on computing the entire electric field and then computing the entire magnetic field as each fields only changes when the other field changes so we should compute the entire fields independently.

If we assume we drop a source into the middle E_y cell, then in the first magnetic pass its adjacent H_z fields will update with non zero values, in the next electric pass the neighbouring E_x and E_y cells will update and so on and so forth propagating the wave outwards from that point.

3.5 Choosing the Algorithm

We chose FDTD over alternative algorithms due to its synergy with an FPGA implementation. An FDFD algorithm would require random access which is not efficient with our BRAM constraints (as we already have a random access renderer) and FFT methods require all other cells to calculate any single cell which is not the most efficient for an FPGA implementation. FDTD provided a solution that was sufficiently accurate while being suitable for parallelisation and our BRAM budget.

3.6 The Coefficients and Their Limits

The coefficient c_a determines how much of a cell's own value it keeps each timestep, so in a vacuum $c_a = 1$ and the field only changes through its neighbours, while $c_a < 1$ models a lossy medium which is what the PML uses. c_b sets how strongly the neighbour difference drives the update and gives us the Courant number $S = |c_b|/2^{13}$, the fraction of a cell the wave moves per iteration. This is bounded above by the CFL stability condition which requires $S \leq 1/\sqrt{2}$ in 2D, as any faster causes the leapfrog scheme to amplify instead of propagate and overflow the Q3.13 range. It is bounded below by quantisation, since c_b is an integer number of Q3.13 steps and the coupling term rounds to zero once it gets too small. We chose $c_b = -717$ ($S \approx 0.0875$), large enough that the wave propagates cleanly while staying well inside the CFL limit.

3.7 Number Format

To prevent segmentation in the BRAM a power of 2 would have been ideal for the number format, so we chose 16 bits as it cuts down from the standard 32 bit Floating Point format while maintaining sufficient accuracy to demonstrate our results. We chose Q3.13 as it covers the range ± 4 (sufficiently large for any constructive interference) while maximising fineness of resolution (0.00012). To tackle the problem of constraining a 32 bit multiplication result back into a 16 bit format we took bits [28:13]. Taking these bits alone truncates towards negative infinity, which biases every multiplication by up to one LSB in the same direction. In the interior $c_a = 1$ leaves nothing to damp this, so the bias accumulates over thousands of iterations into a visible artifact in the visualisation. We fixed this by adding half an LSB before taking the bits, which rounds to nearest and removes the bias, leaving only a negligible truncation error ($\sim 10^{-4}$) and a small error/frame that we deemed a suitable trade off.

Format	Range	Error/frame	Note
Q1.15	± 1	0.04%	range too small, clips on interference
Q2.14	± 2	0.09%	range still too small in edge cases
Q3.13	± 4	0.17%	our choice, enough range and low error
Q4.12	± 8	0.34%	wasted range and double the error
Q3.21	± 4	<0.01%	lower error but 24 bits fragments the BRAM
FP32	huge	$\sim 0\%$	negligible error but double the memory

Table 3.2: The number formats we considered. Error/frame is the rounding error accumulated over one 812 iteration frame.

3.8 Pipelining the Update

This equation is computed in the PL using the following 3 step pipeline:

Stage 1:

$$\delta = B_z^{n+\frac{1}{2}}[i, j] - B_z^{n+\frac{1}{2}}[i-1, j] \quad (3.35)$$

Stage 2:

$$\alpha = c_a \cdot E_x^n[i, j], \quad \beta = c_b \cdot \delta \quad (3.36)$$

Stage 3:

$$E_x^{n+1}[i, j] = \alpha - \beta \quad (3.37)$$

The computation was broken down into a pipeline as it formed a critical path that failed timing in implementation, it still allowed us to have a throughput of 1-2 (depending on the pass) cell per cycle whilst ensuring timing requirements were met. Because each result is written 4 cycles after its read is issued (1 cycle of BRAM read latency plus the 3 pipeline stages), the pipeline must also be drained at the end of each pass: the last 4 results only emerge after the final read, so the solver runs 4 cycles past the cell count to flush them. This makes an iteration $2 \times \text{ROWS} \times \text{COLUMNS} + 4$ cycles, and without this drain the last 4 cells are never written.

3.9 Boundaries and the Absorbing Layer

An issue occurs at the boundary walls of our grid, as there is no more cells beyond them we would intuitively force them to be zero to prevent the accessing of invalid indices. However due to the nature of derivatives this would lead to a reflection of the wave off the boundaries back into the grid as if they were walls, meaning within a few dozen frames the entire grid will become a superposing interfering mess and we would not be able to visualise any interesting experiments.

The industry method to simulate an "infinite plane" to prevent reflections is to use a method called the perfectly matched layer. This method gradually decays the wave near the boundaries to cause negligible reflection. This is done using multiple methods from convolution (CPML) to decay constants that scale linearly or polynomially (UPML). We chose to go with UPML due to its simplicity and accuracy while conserving memory resources, DSPs, LUT usage and cycles. We chose a cubic profile achieving -113 dB as it lead to no visual reflection artifacts.

Cubic conductivity profile with depth d into the absorbing layer:

$$\sigma(d) = \sigma_{\max} \left(\frac{d}{d_{\max}} \right)^3 \quad (3.38)$$

Attenuation coefficient derived from the conductivity:

$$c_a(d) = \frac{2\varepsilon - \sigma(d) \Delta t}{2\varepsilon + \sigma(d) \Delta t} \quad (3.39)$$

At $d = 0$ (interior): $\sigma = 0$, so $c_a = 1$ and the field is unattenuated. The modified update inside the PML:

$$E_x^{n+1}[i, j] = c_a(d) \cdot E_x^n[i, j] - c_b \left(B_z^{n+\frac{1}{2}}[i, j] - B_z^{n+\frac{1}{2}}[i-1, j] \right) \quad (3.40)$$

One-way attenuation after traversing the full PML, where the wave spends $\approx 1/S$ iterations in each cell:

$$A = \prod_{d=0}^{d_{\max}} c_a(d)^{1/S} \quad (3.41)$$

A PML depth of 6 cells was chosen, after experimentation with different depths, because it was the smallest depth that led to unnoticeable reflections while keeping c_a large enough that the PML boundary itself wasn't acting as a wall. Due to the narrow depth of our PML boundaries we chose to store constants in registers (only 18) instead of BRAM as it traded minimal LUT usage for more BRAM availability and simpler logic, especially because we did not need the 1 cycle delay for reading and writing.

Depth d	c_a	c_a (real)
0	8192	1.0000
1	8174	0.9978
2	8045	0.9821
3	7695	0.9393
4	7014	0.8562
5	5892	0.7192

Table 3.3: Cubic attenuation ramp $c_a \approx 8192 - 18.4d^3$

Although the deepest PML cell only multiplies the field by $c_a = 0.7192$, a 28% reduction, this factor is applied on every iteration the wave spends in that cell. Because the wave moves at $S = |c_b|/2^{13} \approx 0.0875$ cells per iteration it takes roughly $1/S \approx 11$ iterations to cross a single cell, so the effective attenuation in that cell alone is $0.7192^{11} \approx 0.02$. Across all six cells of the layer the one-way attenuation is roughly -56 dB, and since the wave reflects off the zeroed outer wall and passes back through, the round-trip attenuation is -113 dB.

3.10 From Three Passes to Two

We chose to parallelise the computation of the Ex and Ey fields reducing our system from 3 pass (Ex, Ey and Bz) to 2 pass (E and B). This was because both Ex and Ey fields only varied on Bz which would have been computed entirely in the pass before and because of our sequential calculation of left to right for Ex and up to down for Ey we could use just 2 read port BRAM with 2 temporary registers to hold the previous Bz values to compute the next Ex and Ey values. This works because computing the new Ex requires Bz[i, j], Bz[i - 1, j] and computing the new Ey requires Bz[i, j], Bz[i, j - 1]. While it might seem that we need 3 read ports the Bz[i, j] value was actually used in the previous calculation so it can just be temporarily held in a register allowing us to make just 2 reads with the 2 read ports available on the single BRAM18 holding the Bz values. For initial conditions [0,0] we assume negative indices are just 0 instead of trying to read from an invalid address which still allows us to only use 2 BRAM ports. This allows us to parallelise Ex and Ey computation without making redundant copies of the BRAM and get an exact 50% performance gain.

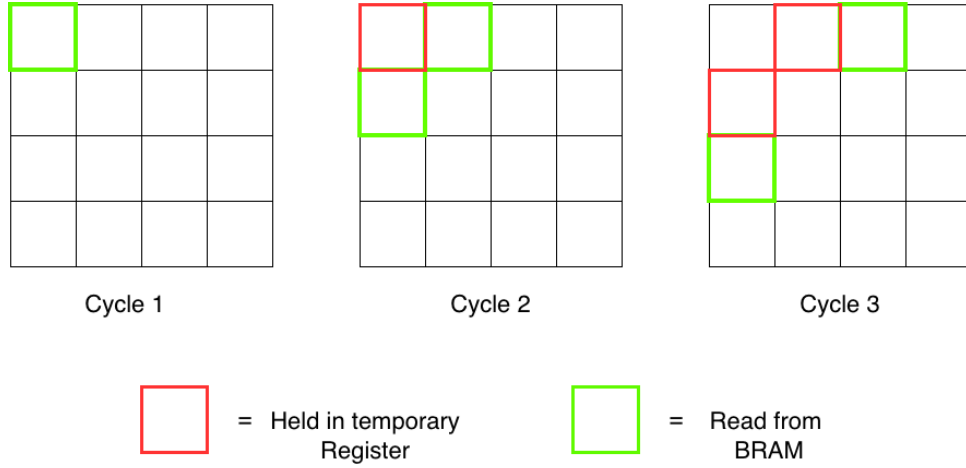


Figure 3.3: Example field updates showing just 2 temporary registers are enough to make at a maximum 2 new Bz reads per cycle

3.11 Lane Parallelism and the Halo

Another optimisation we made was splitting our computation into multiple lanes running in parallel computing different sections of our Yee Lattice. The simplest way to do this was splitting it into horizontal strips although vertical strips would have been equally sufficient. A small Halo would be needed where to compute the first row of the $n+1$ th strip we would need the last values of the n th strip for Bz and Ey, this would ensure that our wave would propagate between the strips.

This works with our limitation of 2 BRAM ports per lane per field. If we take the example of the first and second lane with the first row computation, because all lanes compute together, when the 2nd lane is on its first row the 1st lane is also on its first row, meaning the 1st lane is not using its own 2nd BRAM port as we assume negative indices are 0. This allows the 2nd lane to use the free BRAM port of the 1st lane to read the last row of the 1st lanes strip to "carry" over the wave. This works for the 2nd and 3rd lanes as well as the 2nd lanes own 2nd BRAM port is also free at that moment, allowing the third lane to use it to read the last row in the 2nd lanes strip. This allows us to forgo any stall or redundant copies logic and leads to a direct scaling between number of lanes and increase in performance.

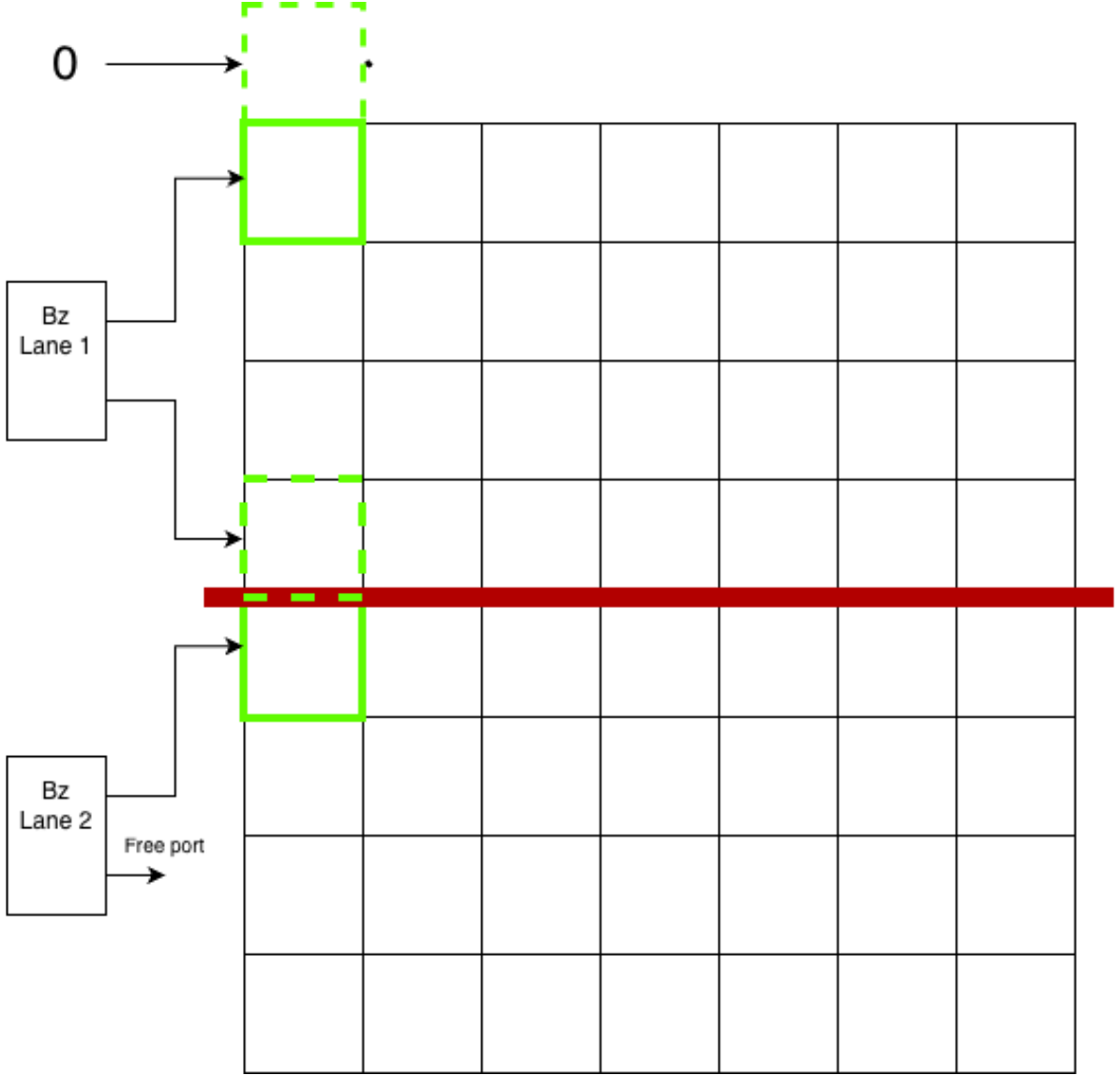


Figure 3.4: Example field update for E_y showing Halo exchange with 2 lane architecture. Solid lines are the 2 cells being updated and dashed line are $Bz[i, j-1]$. The free port can be used for the next lane.

Theoretically we could instantiate however many lanes as we saw fit and it would scale linearly with performance however, this would not necessarily be efficient for BRAM usage. We had to keep in mind that the BRAM on the Pynq Z1 can only be partitioned down to BRAM18s (which actually only store 16 kbits of data) so if we had incredibly small lanes we would have a lot of partially filled BRAM18s whose ports were constantly under load from the FDTD solver not allowing them to use the rest of the memory for other purposes. Therefore we calculated that for a 128x128 grid, the smallest column strip that fit in 1 whole BRAM18 was a 128x8 strip. Another restriction was simply the wavespeed, if we computed it too fast then the wave would move incredibly fast too as there would be more iterations per frame, to combat this we were able to tune the constants in our PML submodule.

This meant that the ideal approach was a 16 lane approach maximising performance and memory tradeoff. This resulted in a 16x speedup compared to our simple 1 lane approach and at 100 MHz we were able to get a throughput of 2.4 billion field updates per second.

This would result in the fields only taking up 24 BRAM36's out of 140 along with a further 5 for calculating S or E and the rest of the 111 BRAM36's for any double buffering or data copies for the rendering engine.

	1 lane	4 lane	16 lane
Grid	64×64	64×64	128×128
Iteration latency	8196 cycles	2052 cycles	2052 cycles
Iterations per 60 fps frame	203	812	812
Field updates /s	150 Million	600 Million	2.4 Billion

Table 3.4: Solver performance scaling with lane count at 100 MHz

3.12 Performance Comparison

As shown in Table 3.4 our 16 lane FDTD solver has a throughput of 2.4 billion field updates per second, to get a reference of how fast that is we tested how many the Pynq's A9 CPU would compute in the same amount of time. As shown in Figure 3.5 there was a 31.7× speedup in the performance of the FDTD algorithm even against an optimised C implementation on the A9 CPU. Furthermore as shown in Table 3.5 the PL is 96 times more power efficient.

config	field updates	vs fpga
1 core scalar	29.8 M/s	80.5x
2 core scalar	58.3 M/s	41.2x
2 core + NEON	75.7 M/s	31.7x
fpga 16 lane	2400.0 M/s	1.0x

Figure 3.5: Solver throughput on the Cortex-A9 against the 16 lane FPGA design

	Throughput	Power	Efficiency
Cortex-A9, 2 cores + NEON	75.7 M/s	~1.5 W	~50 M/s/W
FPGA 16 lane solver	2400 M/s	~0.5 W	~4800 M/s/W

Table 3.5: Field update throughput per watt. CPU throughput measured on the PYNQ-Z1; power figures are the estimated PS load power and Vivado power estimate for the PL respectively

3.13 Verification

Figures 3.6–3.9 show the results of our unit test benches. We prioritised having thorough test benches with comprehensive test cases to make sure when it came time to implement our designs on vivado there would be minimal issues. We made test benches for all the core modules and then top level test benches for the entire FDTD system for 4 and 16 lane to ensure the systems subparts worked before verifying the overall system worked.

```

test 1 passed: d=0 ca=8192 cb=-717
test 2 passed: d=1 ca=8174 cb=-717
test 3 passed: d=2 ca=8045 cb=-717
test 4 passed: d=3 ca=7695 cb=-717
test 5 passed: d=4 ca=7014 cb=-717
test 6 passed: d=5 ca=5892 cb=-717
test 7 passed: d=6 default ca=8192 cb=-717
test 8 passed: ca ramp monotone with depth
all 8 tests passed
tests/tb_pml.sv:102: $finish called at 7000 (1ps)

```

Figure 3.6: PML module tests

```

test 1 passed: iteration took 2052 cycles
test 2 passed: source injected
test 3 passed: top boundary clear
test 4 passed: bottom boundary clear
test 5 passed: left boundary clear
test 6 passed: right boundary clear
test 7 passed: wave crossed lane seam
test 8 passed: done refires
test 9 passed: reset clears state
all 9 tests passed
tests/tb_top_fdd_quad_lane.sv:186: $finish called at 453085000 (1ps)

```

Figure 3.8: 4 lane system tests

```

test 1 passed: iteration took 32772 cycles
test 2 passed: source injected, ey = 8192
test 3 passed: ey boundary rows clear
test 4 passed: ex boundary cols clear
test 5 passed: done refires
test 6 passed: reset halts solver
test 7 passed: pml attenuates, interior untouched (pml=7695 interior=8192)
all 7 tests passed
tests/tb_fdd_solver.sv:248: $finish called at 984415000 (1ps)

```

Figure 3.7: Single solver tests, 128×128

```

test 1 passed: iteration took 2052 cycles
test 2 passed: source injected
test 3 passed: top boundary clear
test 4 passed: bottom boundary clear
test 5 passed: left boundary clear
test 6 passed: right boundary clear
test 7 passed: wave crossed lane seam
test 8 passed: done refires
test 9 passed: reset clears state
all 9 tests passed
tests/tb_top_fdd_hex_lane.sv:216: $finish called at 453085000 (1ps)

```

Figure 3.9: 16 lane system tests

Chapter 4

The Renderer

4.1 Foundations and the baseline design

4.1.1 The rendering problem

The renderer’s task is to turn a two-dimensional field produced by the solver into a shaded three-dimensional image, in real time, on an HDMI display. The field is stored as a *height-field surface*: a grid of values $h(x, y)$ giving the surface elevation at each cell. The renderer casts one ray per screen pixel into this surface and returns the colour that ray sees, so the image is a pure function of pixel coordinates,

$$\text{colour} = f(x, y),$$

evaluated in raster order. Because every pixel’s ray is computed entirely independently of every other, the workload is *embarrassingly parallel*: there is no shared state between pixels, no feedback, and no data dependency across the frame.

Figure 4.1: The renderer within the system: it consumes a field heightmap in on-chip memory and produces a pixel stream for HDMI. Camera and sun parameters arrive from the CPU over AXI-Lite.

4.1.2 The fixed pixel pipeline

Every design in this chapter, from the baseline to the final parallel renderer, is the same four-stage pixel pipeline. What changes between generations is how memory is scheduled and how many pixels are in flight, never the underlying algorithm. The four stages are:

1. **ray_gen** : converts a screen pixel (x, y) into a world-space ray direction D from the camera basis.
2. **marcher** : chains N identical march steps. Each step advances the ray by a fixed increment, looks up the heightmap, and tests whether the ray has crossed the surface. It outputs the hit cell and a hit/miss status.
3. **normal** : estimates the surface normal at the hit from the local height gradient.
4. **shader** : applies diffuse lighting and height colouring to produce (R, G, B) , or sky colour if the ray missed.

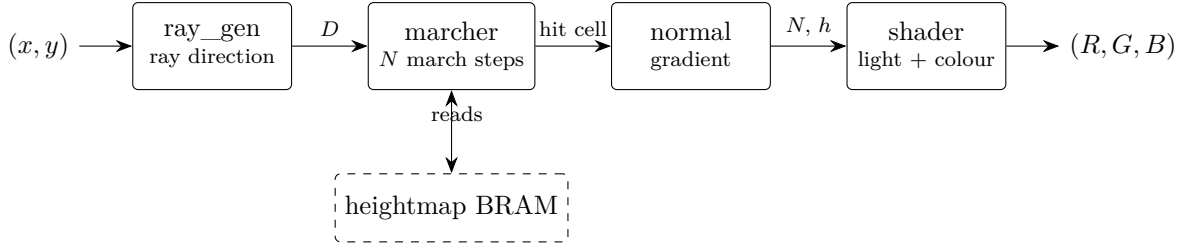


Figure 4.2: The fixed four-stage pixel pipeline, common to every design in this chapter. Later generations modify only the memory scheduling inside **marcher** and the number of pipelines in parallel.

4.1.3 Number representation

The renderer uses fixed-point arithmetic throughout; no floating-point is synthesised. The formats are summarised in Table 4.1. Word length is a substantial design choice: more fractional bits give a smoother surface and fewer staircase artefacts, but widen every multiplier (more DSP) and tighten timing, while more integer bits enlarge the representable world at the cost of precision. The chosen 16-bit **Q2.13** formats place the world in $[-4, 4)$ units with a resolution of $\approx 1.2 \times 10^{-4}$, which is fine enough that the surface looks continuous once interpolation is added (Section 4.2).

Signal	Format	Bits	Role
Position P	Q2.13	16	ray world coordinate
Direction D	Q2.13	16	ray / basis vector
Height h	Q2.13	16	heightmap value
Screen UV u, v	Q1.14	16	centred pixel coordinate
Per-pixel constant K	Q0.15	16	screen-scale factor (= 137)

Table 4.1: Fixed-point formats used across the renderer. All are signed.

4.1.4 Choosing the march depth

Before the architecture can be sized, the march depth must be fixed, because it is the dominant memory cost: each step adds heightmap read ports, and every two reads landing in the same cycle force another dual-port copy of the entire heightmap. Depth is therefore bounded above by the BRAM budget, and the right choice is the *smallest* number of steps N that still captures essentially every surface hit, any step beyond that spends memory for no visual gain.

To locate that floor, the ray generator and march loop were reproduced in fixed-point in Python using the exact **Q2.13** formats, the same camera basis, and the same surface-crossing test as the RTL and run over a representative field surface (two source poles with propagating ripples, matching the solver output). For every pixel, the script records the step index at which its ray first hits the surface. The resulting distribution is shown in Figure 4.3 and the cumulative coverage in Table 4.2.

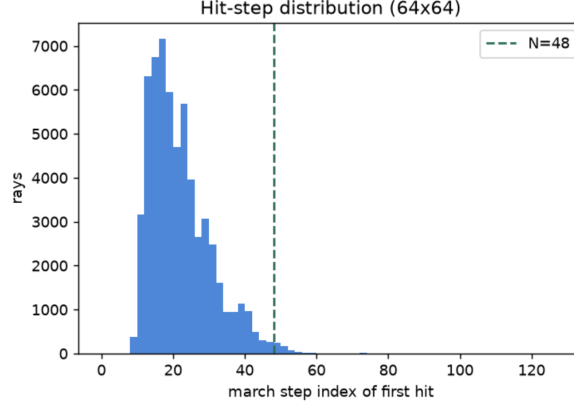


Figure 4.3: Step index at which each ray first hits the surface (64×64 grid, fixed-point march identical to the RTL). The distribution rises near step 10, peaks around step 17, and its tail vanishes by step 48.

N steps	Hits captured
16	33.9%
24	71.0%
32	89.3%
40	96.5%
48	99.2%
64	100.0%

Table 4.2: Cumulative fraction of surface hits captured by march depth N , over the representative field at 64×64 .

Coverage climbs steeply through the knee of the curve and reaches 99.2% at 48 steps; the final 0.8% would cost a further 16 steps and the BRAM that comes with them. The march depth is therefore set to 48 steps for the 64×64 grid, the smallest depth at full coverage.

One detail of the step arithmetic is chosen specifically to save DSPs. Each step advances the ray by $P += DT \cdot D$, a multiply by the step size DT on every axis. By fixing DT to a power of two ($DT = 256 = 2^8$), this multiply becomes a simple bit-shift, costing no DSP at all. With DT an arbitrary value, each step would need three multipliers for the position advance : $48 \times 3 = 144$ extra DSP across the marcher, so the power-of-two choice leaves only the interpolation itself as the per-step multiplier cost.

4.1.5 The baseline architecture

With the depth fixed, the baseline establishes the skeleton on which every later design is built. It marches 48 steps over a 64×64 heightmap, running at one pixel per clock cycle in a single clock domain locked to the pixel clock.

Each march step issues one heightmap read, it samples the single nearest grid cell, so 48 steps need 48 read ports. The normal stage reads four neighbouring cells to form a central-difference gradient, adding four more. The baseline therefore exposes $48 + 4 = 52$ read ports. A Xilinx RAMB36 block is dual-port, serving two reads per cycle, so these map to **26 BRAM tiles**. The pipeline latency is $4 + (48 \times 4) + 4 + 1 = 201$ cycles, after which it sustains one finished pixel every cycle.

At 26 tiles this baseline already consumes $26/140 = 18.6\%$ of the device’s BRAM budget, before any interpolation is added.

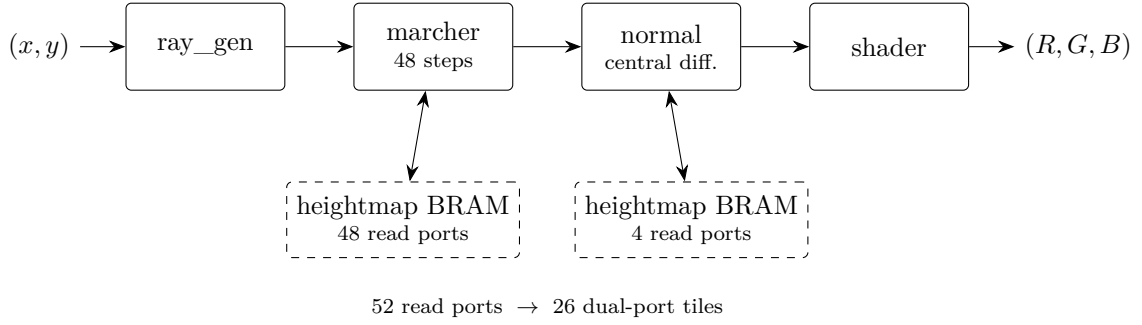


Figure 4.4: Baseline renderer. Forty-eight marcher ports plus four central-difference normal ports give 52 read ports (26 BRAM tiles).

Because hit detection and shading both use the single nearest sample, the baseline image is visibly blocky (Figure 4.5). Removing that blockiness is the subject of the next two sections; the baseline’s role is to fix the interfaces, the fixed-point formats, and the pipeline timing that every later design inherits unchanged.

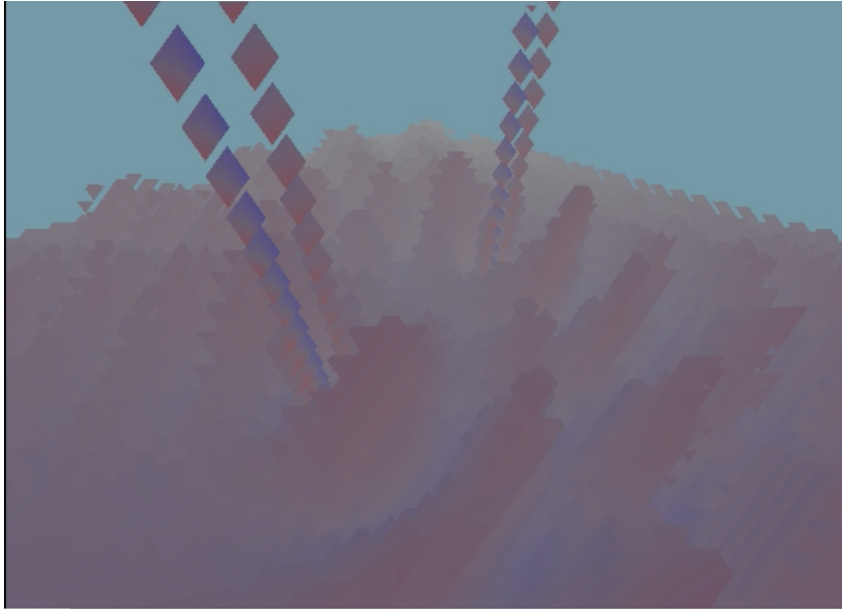


Figure 4.5: Baseline output. The nearest-sample surface renders as discrete plateaus with hard cell-boundary edges that motivates interpolation.

4.2 Shading interpolation: the free visual win

The baseline’s blockiness has two separate causes, and they are worth separating because they have very different costs to fix. The *silhouette*, where terrain meets sky, is blocky because the marcher stops on whole cells. The *shading*, the lighting and colour across each visible face, is blocky because the normal and the surface height are taken from a single nearest cell, so every cell renders as one flat-shaded plateau. This section fixes the second cause, and does so at *zero* additional BRAM, which is why it comes first: it is the cheapest visual improvement available.

4.2.1 Reusing the reads the normal already makes

The insight is that the normal stage already pays for four heightmap reads, and those four reads contain everything a smooth surface needs. In the baseline the normal reads four neighbours in a “plus” pattern ($ix \pm 1, iy \pm 1$) to form a central difference. If instead it reads the four *corners* of the cell the ray landed in,

$$h_{00} = (ix, iy), \quad h_{10} = (ix+1, iy), \quad h_{01} = (ix, iy+1), \quad h_{11} = (ix+1, iy+1),$$

then the same four values serve two purposes at once: a forward-difference normal and a bilinearly-interpolated surface height. The read count is unchanged, still four ports, still the same two BRAM tiles, so the improvement is free in memory.

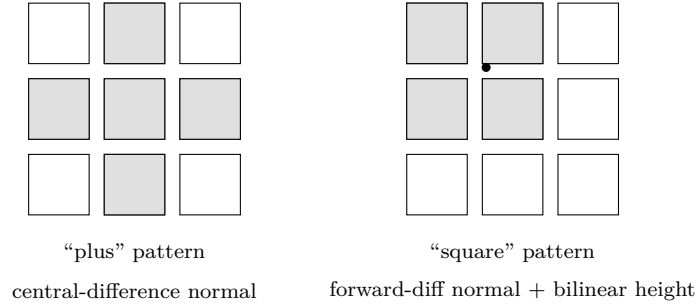


Figure 4.6: The baseline reads four axis neighbours for a central-difference normal (left). Design 1 reads the four corners of the hit cell instead (right), serving both a forward-difference normal and a bilinear surface height from the same four reads and no extra ports.

4.2.2 The interpolation mathematics

The primitive underneath everything here is linear interpolation (a “lerp”): given two corner heights and a fractional position between them, estimate the height on the straight line joining them. Bilinear interpolation extends this to a cell by applying it three times, two lerps along the top and bottom edges in x , then one lerp between those results in y , recovering a smooth surface height h_{interp} from the four corners $h_{00}, h_{10}, h_{01}, h_{11}$ at the ray’s sub-cell position (x_f, y_f) . Figure 4.7 shows the construction.

Figure 4.7: Bilinear interpolation as three lerps over the hit cell: two edge lerps in x give h_{top} and h_{bot} , and a final lerp in y gives the smooth surface height h_{interp} .

Each lerp is one subtract, one multiply by the 8-bit fraction (a single DSP slice), and one add, kept entirely in fixed point.

Parallelism : On a CPU these operations are inherently sequential: the two x -lerps and the forward differences would issue one after another, and the final y -lerp would wait on both x -lerps, costing several dependent instructions per pixel. In the FPGA the computation is laid out spatially instead of temporally. The two x -lerps h_{top} and h_{bot} are independent, so they are placed in the same pipeline stage and their multipliers run in the same clock cycle; the forward differences N_x and N_y are computed in that same stage on their own adders.

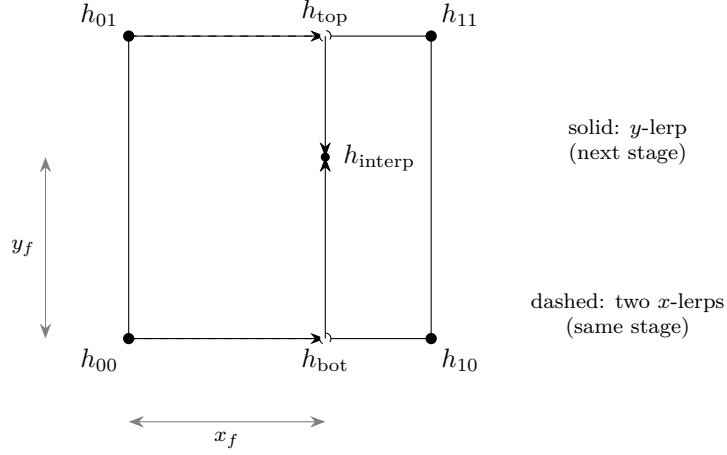
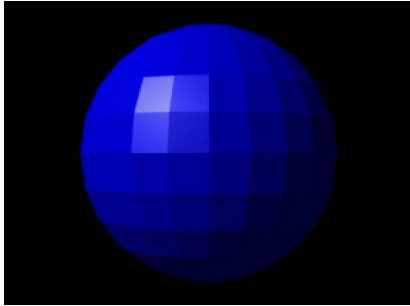


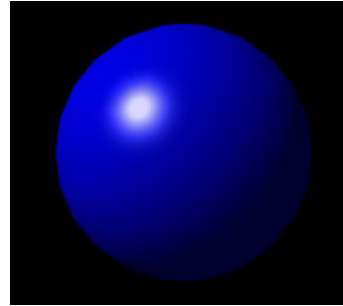
Figure 4.8: Bilinear interpolation as three lerps over the hit cell: two edge lerps in x give h_{top} and h_{bot} , and a final lerp in y gives the smooth surface height h_{interp} .

4.2.3 Cost

The change adds one pipeline stage to the normal, the final y -lerp takes its latency from four cycles to five, and a small amount of logic: three lerp multipliers and two forward-difference subtractions. No BRAM-heavy structure is introduced, and the $48 + 4 = 52$ read ports (**26 tiles**, $26/140 = 18.6\%$) are unchanged from the baseline.



(a) Flat (nearest-sample) shading.



(b) Design 1 bilinear shading.

Figure 4.9: Flat-shaded baseline (left) versus Design 1 shading interpolation (right). The per-cell facets disappear and the lighting varies smoothly across the surface; the silhouette is unchanged.

4.3 Folding and the FIFO scanout

Folding the renderer to save BRAM has a consequence beyond throughput: it changes the clock domain the renderer must live in. At one pixel per cycle the renderer could be clocked directly at the display's pixel rate, but a folded renderer produces pixels more slowly, so to keep the display fed it must run in a faster clock domain than the HDMI output. The two domains must then be bridged. This section describes the video pipeline that performs that bridge, the clocking, the timing generation, and the asynchronous FIFO that carries pixels across the domain boundary, together with the reliability measures that keep the display locked.

4.3.1 Clocking and the two domains

The HDMI output was implemented as a reusable PYNQ-Z1 video pipeline around the ray-rendering core. The board's 125 MHz PL clock is converted using a Vivado Clocking Wizard into a 25 MHz pixel clock for 640×480 VGA timing, a 125 MHz serial clock for TMDS generation, and a 100 MHz renderer clock. A timing module generates the full 800×525 VGA raster, including blanking, HSYNC, VSYNC, and active-video enable. The renderer operates in the 100 MHz domain, while HDMI scan-out operates in the 25 MHz domain, so rendered RGB pixels cross through an asynchronous FIFO before being passed to the Digilent `rgb2dvi` IP, which drives the differential HDMI TMDS signals.

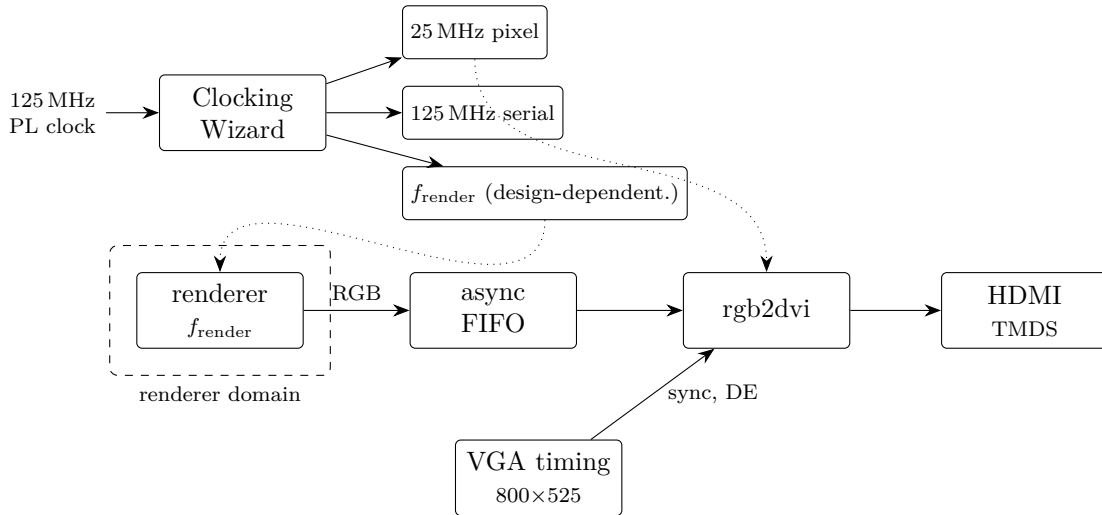


Figure 4.10: The HDMI video pipeline. The Clocking Wizard derives the 25 MHz pixel and 125 MHz serial clocks from the 125 MHz PL clock, plus a renderer clock f_{renderer} whose value depends on the design's fold factor. Rendered pixels cross an asynchronous FIFO from the renderer domain into the 25 MHz pixel domain before `rgb2dvi` drives the TMDS outputs.

4.3.2 Matching throughput and latency to the display

Different renderer versions can use the same HDMI pipeline as long as their throughput matches the fixed display rate. HDMI requires one pixel every 40 ns during active video; for example, the 48-step Design 4 renderer produces one pixel every four 100 MHz cycles, matching this rate. Pipeline latency is handled by delaying HSYNC, VSYNC, and data-enable until they align with the corresponding RGB result, while throughput is handled by the renderer schedule and FIFO. The FIFO absorbs clock-domain phase differences and short stalls, but it cannot compensate for a renderer that is consistently slower than the HDMI pixel rate. Therefore, each design must be checked using both latency and initiation interval: latency determines the required video-control delay, and initiation interval determines whether active video can be continuously filled.

4.3.3 Reliability

A reliability fix was added so `vid_pVDE` remains tied to display timing rather than FIFO availability; if the FIFO underflows, magenta diagnostic pixels are shown instead of dropping HDMI signal lock.

4.3.4 The limit of the FIFO scanout

The FIFO works only while the renderer can be clocked fast enough to deliver one pixel every 40 ns of active video on average. For a renderer folded to one pixel every F cycles, sustaining the 25 Mpix/s active rate requires a PL clock of

$$f_{\text{PL}} = F \times 25 \text{ MHz},$$

since each pixel now takes F clock cycles instead of one. Table 4.3 evaluates this for each fold factor against the 125 MHz limit.

Fold F	Throughput	Required f_{PL}	Feasible ($\leq 125 \text{ MHz}$)?
1	1 px / cycle	25 MHz	yes
2	1 px / 2 cyc	50 MHz	yes
4	1 px / 4 cyc	100 MHz	yes
8	1 px / 8 cyc	200 MHz	no
16	1 px / 16 cyc	400 MHz	no

Table 4.3: PL clock required to sustain the 25 Mpix/s active display rate for each fold factor, with a direct FIFO scanout. Beyond a four-cycle fold the required clock exceeds the device’s limit.

At one pixel every four cycles the required 100 MHz is met, this is exactly the clock at which Design 4 runs. Folding any further, however, demands 200 MHz at eight cycles and 400 MHz at sixteen, both well beyond the 125 MHz the PL can support. The FIFO scanout is therefore hard-limited to a four-cycle fold: it cannot exploit the deeper folding that would save the most BRAM, because there is no clock fast enough to keep the display fed. Breaking this limit requires decoupling the renderer from the display entirely, which is the subject of Section 4.6.

4.4 Silhouette interpolation and the folded board design

Shading interpolation smoothed the lit surface but left the silhouette, the boundary between terrain and sky, on cell edges, because the marcher still stops at the nearest whole cell. Smoothing that boundary is the one improvement that genuinely cannot be made for free: it requires the marcher itself to interpolate at every step, not just once at the hit. This section first shows why that is so expensive (Design 3), then how folding buys the cost back and produces the board-verified renderer (Design 4).

4.4.1 Design 3 : bilinear in the marcher, and the cost explosion

To make the ray stop on the smooth surface rather than on a cell boundary, each march step must test the ray against the bilinearly-interpolated height, not the nearest sample. That means every step now needs the four corners of the cell it is in : four heightmap reads per step instead of one.

The consequence for memory is severe. With four corner reads at every step, 48 steps perform $48 \times 4 = 192$ heightmap reads per pixel. Design 3 spreads these over a two-cycle window (half-rate), giving $192/2 = 96$ reads per cycle. A dual-port RAMB36 serves two reads per cycle, so this needs $96/2 = 48$ heightmap copies; at $64 \times 64 \times 16$ -bit each copy spans two tiles, for **96 tiles** ($96/140 = 68.6\%$).

Moreover, the renderer does not have the full BRAM budget to itself: the solver team's FDTD engine also consumes RAMB36 tiles for its field storage, so the renderer's true ceiling is well below 140. This makes the cost decisive, scaling the heightmap to 128×128 , which would render the propagating wave far more clearly, multiplies each copy's depth from two tiles to eight and is simply not feasible in this architecture.

4.4.2 Folding the reads : Design 4

Design 4 changes nothing about the mathematics or the image; it changes only *when* the reads happen. Instead of four corner reads over two cycles on two ports, each step issues its four reads one at a time over *four* cycles on a *single* port:

$$\text{Design 3: } \frac{4 \text{ reads}}{2 \text{ cycles}} \times 2 \text{ ports} \longrightarrow \text{Design 4: } \frac{4 \text{ reads}}{4 \text{ cycles}} \times 1 \text{ port.}$$

The total reads per pixel are identical, they are simply spread more thinly in time. Folding the same 192 reads over a *four*-cycle window gives $192/4 = 48$ reads per cycle, served by $48/2 = 24$ dual-port copies, or **48 tiles**. On top of this, Design 4 forwards the four hit-cell corners the marcher has already read directly into the normal stage, so the normal no longer reads the heightmap at all and adds no tiles of its own. Design 4 therefore renders the smooth silhouette in **48 tiles** ($48/140 = 34.3\%$), with the saving bought purely through throughput: one pixel every four cycles instead of every two.

Even at one pixel every four cycles, the renderer still produces a frame faster than the display can show it: at 100 MHz this is 25 Mpix/s, or ~ 81 fps at 640×480 , comfortably above the 60 fps HDMI cap, so the throughput traded away for BRAM costs nothing in perceived performance.

4.4.3 Folding the multipliers: from 70% to 26% DSP

The idle time that folding creates can be exploited a second time, on the multipliers. Each march step's bilinear interpolation uses three multiplies; unrolled across 48 steps that is $48 \times 3 = 144$ DSP, which with the ray-generation and shading stages totals roughly 150, about 70% of the device's 220 DSP48E1 slices. This is the same kind of waste folding already removed from the memory: at one pixel every four cycles, each step's multiplier is busy only one cycle in four and idle the other three.

The fix is identical in spirit to the BRAM fold. The three lerps of a step are cascaded into sub-stages that time-share a single DSP across the four-cycle window, so one multiplier does the work the three previously did. This collapses the marcher from 144 to 48 DSP. Including ray-generation and shading, the whole renderer now uses just **58 DSP** ($58/220 = 26.4\%$), down from 70%.

The two post-implementation reports in Figure 4.11 confirm this directly: between the parallel and the time-shared builds every resource is unchanged except DSP, which falls from 70% to 26%. The BRAM figure holds at the 35% (49/140 tiles) predicted by the folding analysis, just as expected.

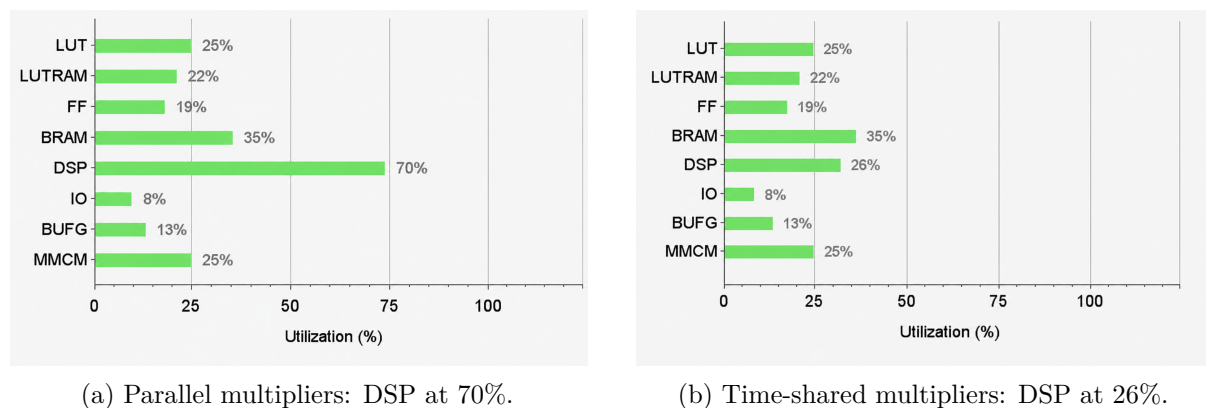


Figure 4.11: Vivado post-implementation utilisation before (left) and after (right) folding the multipliers. Every resource is identical except DSP, which drops from 70% to 26%; BRAM holds at the predicted 35% (49/140 tiles).

The design closes timing at 100 MHz with positive setup and hold slack, as summarised in Table 4.4.

Metric	Value
Setup slack (WNS)	+0.457 ns
Hold slack (WHS)	+0.016 ns
Core clock	100 MHz

Table 4.4: Design 4 timing closure on the PYNQ-Z1: positive setup and hold slack at 100 MHz.

4.5 Result :

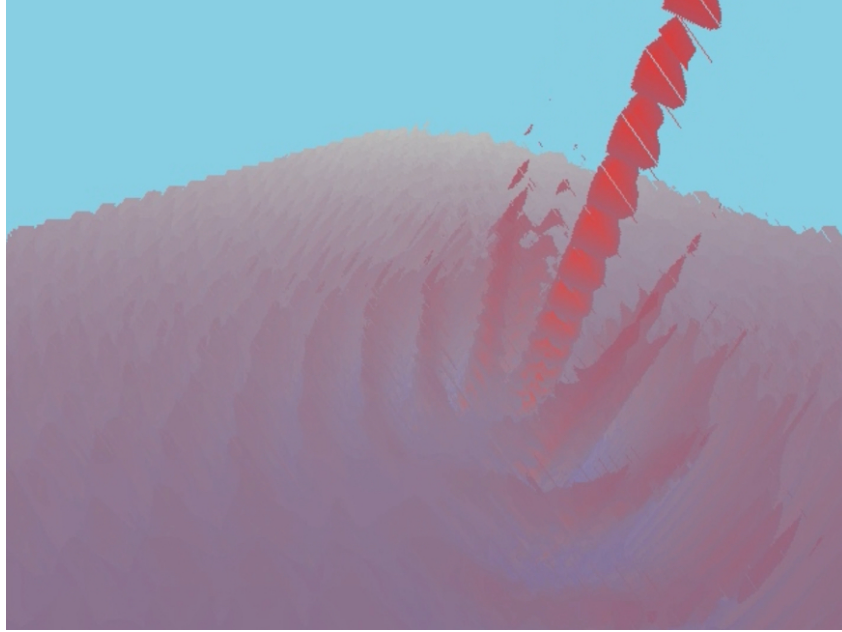


Figure 4.12: Output of the board-verified Design 4 renderer (64×64 field): the source pole and its propagating ripples are rendered with smooth shading and a smooth interpolated silhouette, on hardware at 640×480 .

4.5.1 Scaling to 128x128 is still not possible without parallel lanes

The same calculation rules out scaling to 128×128 , however. A $128 \times 128 \times 16$ -bit heightmap is 256 Kibit, so each copy now spans $\lceil 256/36 \rceil = 8$ tiles instead of two. Keeping 48 steps and the four-cycle fold still needs 24 copies, but at eight tiles each that is $24 \times 8 = \mathbf{192 \text{ tiles}}$ ($192/140 = 137\%$), and the solver's own BRAM pushes the true ceiling lower still. Even Design 4's folding therefore cannot reach the resolution that would show the propagating wave clearly, which is the limit the next sections set out to break.

4.6 The frame buffer and parallel lanes

The natural next goal is to scale to a larger heightmap, so the propagating wave is shown in far greater detail. The obstacle is not the renderer's logic but its output path. With the direct-to-HDMI scheme, the renderer must deliver one active pixel every pixel-clock to keep the display fed, which ties its throughput rigidly to the 25 Mpix/s active rate. A larger, deeper render needs more cycles per pixel, and to still sustain 25 Mpix/s the renderer clock would have to rise in proportion, the deeper folds 1 pixel every 8 cycles and 1 pixel every 16 cycles require clocks running at respectively 200MHz and 400MHz, all far beyond the xc7z020's 125 MHz support limit. The direct path simply cannot clock fast enough to scale.

The way out is to break the rigid link between render rate and display rate. This section does so with two changes: decouple the renderer from the display using a frame buffer, then recover the frame rate with parallel lanes rather than a faster clock.

4.6.1 Decoupling with a frame buffer

The fix is to stop feeding HDMI directly. Instead the renderer writes completed pixels into a full-frame buffer in DDR, and the HDMI side reads that buffer back at its own constant 25 MHz, independent of how fast the renderer fills it. Producer and consumer are now decoupled: the renderer may run as slowly as it likes. Two buffers are used in alternation : the renderer writes one while HDMI scans the other, swapped at each frame boundary so the display never reads a half-written frame .

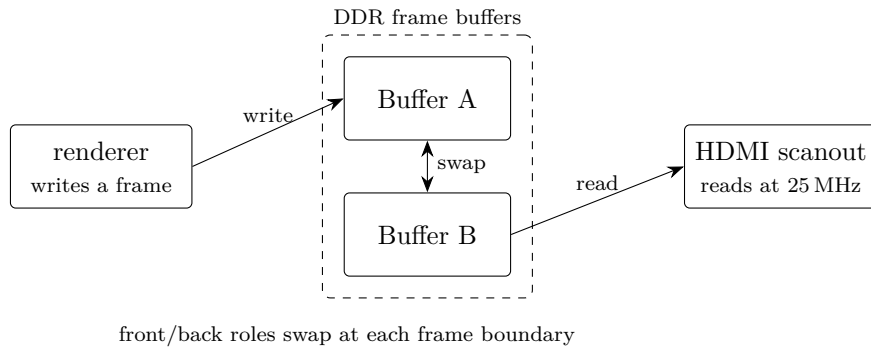


Figure 4.13: The renderer writes one frame buffer in DDR while HDMI scans the other at a constant 25 MHz; the buffers swap at each frame boundary. Producer and consumer rates are fully decoupled.

Once the renderer is no longer rate-locked to the display, it is free to fold as deeply as it likes to save BRAM.

4.6.3 Why this uses fewer resources

The decisive point is that three deeply-folded lanes cost *less* memory than one lightly-folded lane, not more. A single lane at the four-cycle fold spreads its reads over four cycles and needs a certain number of heightmap copies; folding sixteen cycles instead spreads the same reads four times more thinly, cutting each lane’s copies by four. Three such lanes therefore use $3 \times \frac{1}{4} = \frac{3}{4}$ of the single-lane copy count, fewer copies overall, while running at 60 fps through the frame buffer. Parallelism at a deep fold is strictly cheaper in BRAM than a single lane at a shallow fold.

This is the headroom the whole chapter has been building toward: a renderer that hits the display’s frame-rate ceiling using fewer memory copies than the board-verified single lane, leaving budget to spend on the quality trade-offs such as larger grid, more steps, smoother silhouette that the next section weighs.

4.7 Final comparison: five candidate designs

The frame buffer and parallel lanes remove the frame-rate ceiling, but they do not remove the resource ceiling. Scaling to 128×128 raises a new problem: the step size DT scales with the grid, so at 128×128 each step covers half the world distance it did at 64×64 , and the ray needs roughly twice as many steps to reach the same distance. Rerunning the hit-coverage analysis of Section 4.1.4 at 128×128 confirms this, 48 steps now cover only 87% of hits, and reaching the 99% knee would take about 80 steps. The design the project would most like to build, three lanes, 80 steps for full coverage, bilinear silhouette, 128×128 , 16-bit heights, is far out of budget on both BRAM and DSP. Something must be traded away.

Each advantage developed in this chapter :parallel lanes, full hit coverage, the bilinear silhouette, the full 128×128 grid, and 16-bit height precision can be relaxed in turn, giving five candidate designs. They are compared in Table 4.5.

Design	Lanes	Steps	Silhouette	Grid	Height	Tiles	Coverage
1 · fewer steps	3	48	bilinear	128^2	16-bit	72	87%
2 · drop silhouette	3	64	nearest	128^2	16-bit	48	97%
3 · small grid	3	64	bilinear	64^2	16-bit	48	100%
4 · two-lane full	2	64	bilinear	128^2	16-bit	128	97%
5 · 8-bit height	3	64	bilinear	128^2	8-bit	96	97%

Table 4.5: Five candidate designs. Each keeps four advantages and relaxes one. Tiles are total RAMB36 count; coverage is the fraction of surface hits captured at 128×128 . Designs 1–3 and 5 run at 61 fps; design 4 at 41 fps.

Design 5 : halve the height precision. The other four designs each give up something the eye notices, coverage, edge smoothness, detail, or frame rate. Design 5 instead attacks the cost at its source: the size of a heightmap entry. Storing each height in 8 bits rather than 16 shrinks a copy from 8 tiles to 4, so the full design :three lanes, 64 steps, bilinear silhouette, 128×128 , 61 fps, 97% coverage costs

$$3 \times 8 \times 4 = 96 \text{ tiles } (96/140 = 68.6\%),$$

and keeps every other advantage intact. Its price is height precision: 8 bits gives 256 levels rather than 65,536. This is the least visible of the five trades, indeed the rendered surface varies smoothly, and the gap between two adjacent height levels is far below what the eye can resolve against the display’s own colour quantisation which is why Design 5 is chosen for the final renderer.

4.7.1 Fitting the DSP budget

BRAM is not the only resource three lanes multiply. Each march step’s bilinear interpolation, after the time-sharing of Section 4.4, uses one DSP, so a 64-step lane spends 64 DSP in its marcher; with ray-generation and shading the single lane would sit near 74 DSP, and three lanes at that rate would need 222 DSP just over the device’s 220.

The shading stage is optimised to close this gap. By fixing the sun at a constant direction, the diffuse lighting dot-product becomes a multiplication by *constants* rather than by runtime values; constant multipliers synthesise into shift-and-add logic in the fabric rather than DSP slices, cutting the per-lane ray-generation and shading overhead to 6 DSP. The whole renderer then uses

$$3 \times (64 + 6) = 210 \text{ DSP } (210/220 = 95.5\%),$$

fitting within budget.

4.7.2 The final design

The final renderer is three lanes at 61 fps, a 64-step bilinear march over a 128×128 field with 8-bit heights and a fixed sun, reaching 97% hit coverage. It occupies 96 BRAM tiles and 210 DSP slices. Together with the solver's 29 tiles, this places 125 of the device's 140 BRAM tiles (89.3%) and 210 of its 220 DSP slices (95.5%) in use, the design is fitted closely to the hardware it runs on.

Chapter 5

Interface and System Integration

The previous section described how a single FDTD cell is updated. This section is about everything around that: how the solver, the wave source, the memory buffers and the renderer are wired together into one working design on the PYNQ-Z1, how the processor drives it all live, and what the finished system actually shows on screen. The solver itself is treated here as a component with a clean interface rather than re-explained.

5.1 System Overview

At the top level the programmable logic is one long pipeline (Fig. 5.1). The source generator feeds the FDTD solver, the solver’s output is reduced to a per-cell field magnitude, that magnitude is held in a double buffer, a bridge copies it into the renderer’s memory, and the renderer finally draws it out over HDMI. The guiding idea throughout is that each stage is loosely coupled to its neighbours: because the solver and the display naturally run at different and unrelated speeds, we never let one stage read another’s half-finished work. All field values are stored in signed Q3.13, and the whole field datapath shares a single 25 MHz clock so that there are no clock-domain crossings on the path where data integrity matters most. Only the 3D renderer runs faster, at 100 MHz, and we bring its pixels back into the 25 MHz domain through an asynchronous FIFO.

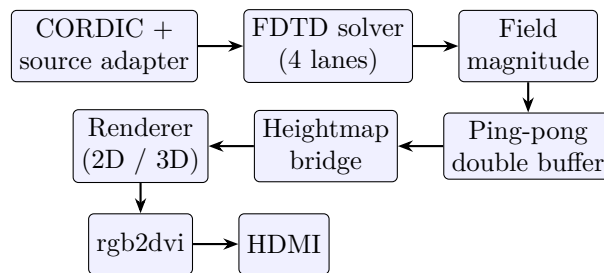


Figure 5.1: The end-to-end pipeline. Everything from the source generator to the bridge shares one 25 MHz clock; the renderer and HDMI serialiser follow on.

5.2 Building the Design from a Script

Rather than wiring the design together by hand in Vivado’s graphical block-design editor, we generate the whole thing from a Tcl script. The reason is the same one that leads any software project to version-control its source rather than its binaries: the block-design file Vivado produces is a large, auto-generated XML blob that cannot sensibly be reviewed or merged, whereas a script is readable, diffable, and regenerates an identical bitstream every time. We therefore

commit the HDL and the script and treat them as the single source of truth, which also makes it cheap to keep several design variants that differ by only a few lines.

The script runs in the phases of Fig. 5.2. Our own modules are first packaged as Vivado IP so they can be dropped onto the canvas; the renderer is instead added as a *module reference*, which has the quirk of requiring a Verilog (not SystemVerilog) top-level wrapper. One lesson worth recording is that we give every peripheral an explicit fixed address: we originally relied on Vivado’s automatic assignment, which orders peripherals alphabetically, and discovered that simply adding new blocks silently shifted all the existing ones and broke the addresses the notebook was written against.

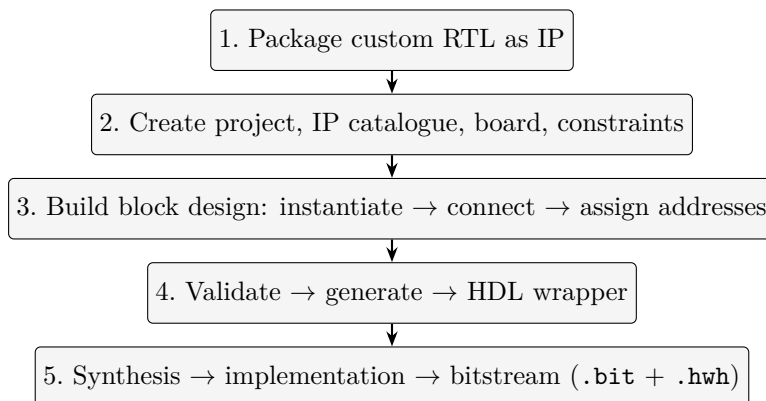


Figure 5.2: The scripted build. Phases 1–4 always run; phase 5 is switched on by an environment variable, so the same script builds either just the project or the full bitstream.

5.3 Clocking

Every clock in the design comes from a single mixed-mode clock manager — the clocking wizard of Fig. 5.3 — fed by the board’s 125 MHz reference (Table 5.1). Each domain has its own reset that is held until the wizard asserts its `locked` output, so no block starts running before its clock is stable. Where two domains have to meet we handle the crossing deliberately rather than hoping for the best: an asynchronous FIFO carries pixels from the 100 MHz renderer into the 25 MHz scan-out, a dual-clock block RAM holds the heightmap written on one clock and read on another, and the handful of control strobes that cross from the processor’s clock are passed through proper synchronisers.

Clock	Use
25 MHz	field datapath, 2D renderer, video timing
100 MHz	3D ray-march renderer core
125 MHz	HDMI / TMDS serialiser
50 MHz	processor AXI register bus

Table 5.1: Clock domains derived from the 125 MHz board reference.

5.4 Generating the Source

The wave is launched by a sinusoidal point source built from the two blocks in Fig. 5.4. A Xilinx CORDIC core does the trigonometry — it takes a phase angle on its input stream and returns the sine and cosine — and our `cordic_source_adapter` wraps it into something the rest of the system can use. The adapter accumulates the phase (advancing it one step per iteration,

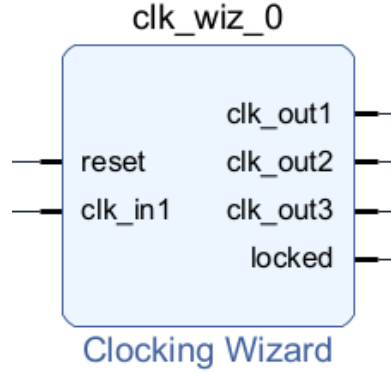


Figure 5.3: The cloning wizard: one 125 MHz input becomes the three on-chip clocks, with a `locked` output that holds the resets until they are stable.

which is what sets the source frequency), drives the AXI-Stream handshake to and from the CORDIC, and scales the result by an amplitude the processor can change on the fly. It can also inject the *difference* of successive samples rather than the raw sine: this carries no average value, which matters because a moving source otherwise leaves a permanent “trail” of deposited energy behind it.

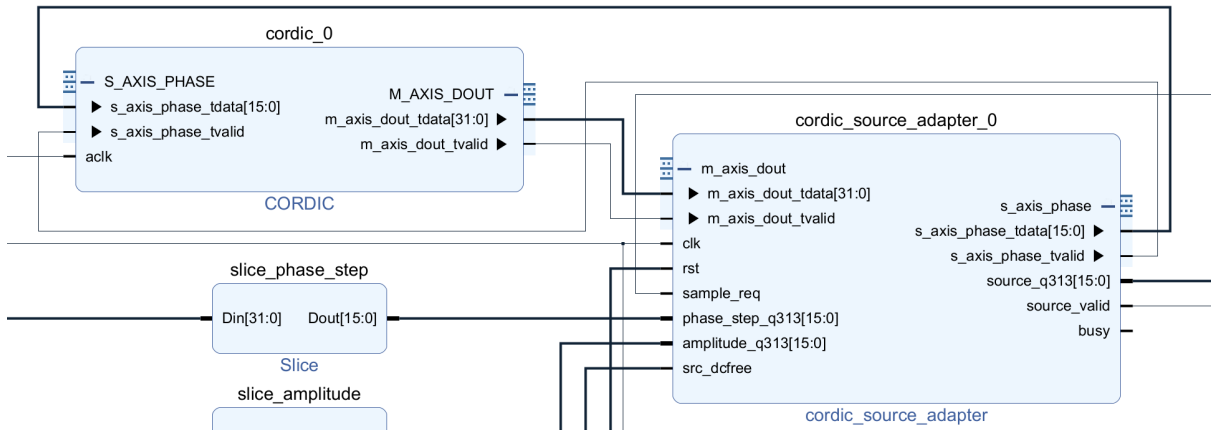


Figure 5.4: Block design of the source: the CORDIC core (left) feeds sine/cosine over an AXI-Stream link into `cordic_source_adapter`, which the slice blocks supply with the amplitude and phase-step set by the processor.

5.5 The Moving Source and the Doppler Effect

The system can also show a Doppler effect. If the source is made to travel through the grid, the wavefronts compress ahead of it and stretch out behind, in the same way a passing siren sounds higher in front and lower behind; past the (numerical) wave speed the fronts can no longer move out of the way in time and form a Mach cone. Producing this cleanly is as much an architectural decision as a physical one, and it is a useful example of how work is divided between the processor and the fabric.

The effect depends entirely on the source moving *smoothly*: it advances only a small fraction of a cell on each FDTD iteration — we keep the position to a resolution of $1/256$ of a cell — and it has to do so in lockstep with the solver, one small step for every iteration completed. This is precisely what rules out driving the motion from the processor. The PS reaches the fabric only

over the AXI register bus through a Python loop, on a millisecond timescale, whereas the solver finishes an iteration in microseconds; if the PS tried to move the source itself it could manage only a few coarse jumps per second, too slow and too granular, and the even wavefront spacing that produces the Doppler effect would no longer be smooth.

The moving source therefore lives in the programmable logic. A small position generator inside the FDTD adapter keeps the source coordinate in a fixed-point accumulator, adds the velocity to it once per completed iteration, and reflects it off the PML walls so the source bounces around inside the grid rather than walking off the edge. The processor is left with the part it is actually suited to: through the slice registers of Fig. 5.9 it sets the velocity, arms the motion, and — because the real hardware runs far faster than is comfortable to watch — throttles the iteration rate so the sweep is slow enough to see. The PS decides *what* the source should do; the PL is what actually does it, iteration by iteration. That division of labour — fast, regular work in the fabric and slow, high-level choreography from the processor — is the same pattern that recurs throughout the design.

5.6 Decoupling the Solver and the Display

This stage resolves a timing mismatch between the solver and the display. The solver finishes a new frame whenever it happens to — a rate that depends on the grid size and how much we have slowed it for viewing — while the renderer has no such freedom: it must read a frame every twenty-fifth of a second to feed the display. If we simply pointed the renderer at the memory the solver was filling, it would now and then catch a frame half-written, with the top of the screen showing the new frame and the bottom the old one. This is the textbook producer-consumer problem, and we use the textbook answer: two identical memories and a small controller (Fig. 5.6). At any moment one memory is being written by the solver and the other is being read by the renderer; when the solver signals that a frame is complete, the controller swaps the two roles in a single clock cycle, as sketched in Fig. 5.5. Because the swap is atomic the renderer always sees a whole, self-consistent frame, and because each memory is dual-ported the reading and writing never collide. As both sides live in the same 25 MHz clock, none of this needs clock-crossing logic.

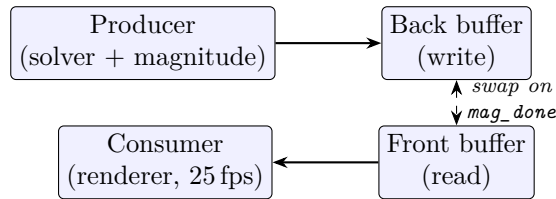


Figure 5.5: The double-buffering idea: the producer fills the back buffer while the consumer reads the front, and the two swap atomically each frame.

5.7 The Bridge to the Renderer

The renderer reads its terrain from its own private heightmap memories, so a bridge (Fig. 5.7) copies each finished frame across. It does this only during the display’s vertical blanking interval, when the renderer is issuing no reads, so the renderer can never catch the heightmap mid-update — the same tear-free principle as the double buffer, applied a second time. On the way across it downsamples the 128×128 field to the 64×64 heightmap and applies a height scale the processor can adjust live.

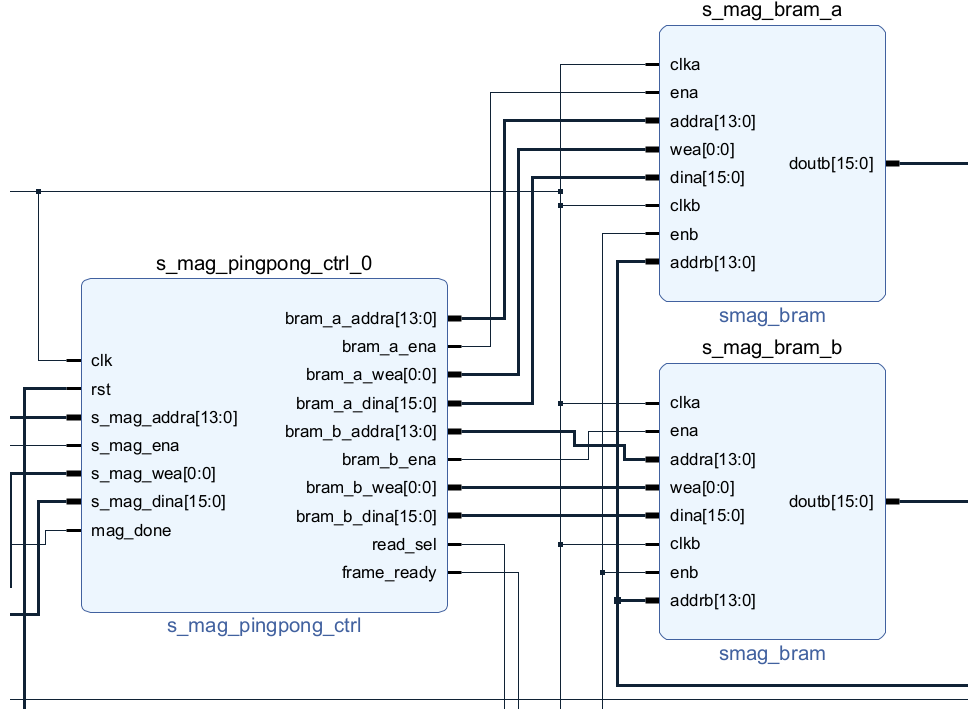


Figure 5.6: The same scheme in the block design: `s_mag_pingpong_ctrl` routes the solver’s writes to whichever of the two `s_mag` memories is the current back buffer, and exports `read_sel/frame_ready` to the reader.

5.8 Driving It from the Processor

Everything the operator touches goes through the processor’s AXI bus (Fig. 5.8): the PS7 processing system drives an interconnect that fans out to a set of simple GPIO register blocks, alongside a reset controller that ties the whole AXI domain to the same locked clock. Because each GPIO register is 32 bits wide but usually carries several smaller values, slice blocks (Fig. 5.9) split each register into the individual fields the hardware consumes, and concatenation blocks gather status bits back into the read-back word. The resulting map is given in Table 5.2. Exposing the controls this way is what makes the demonstration interactive: the operator changes a value in the notebook and the simulation responds immediately, with no rebuild.

Address	Function
0x41200000	control: amplitude, frequency, source position, mode, clear
0x41210000	motion: source velocity, speed throttle, camera load
0x41220000	status (read-only): checksum and flags
0x41230000–0x41250000	camera basis (3D build)

Table 5.2: The AXI-GPIO register map exposed to the notebook.

5.9 Multi-Lane Parallelism

To speed the solver up we split the grid into four horizontal *lanes* and update them in lockstep, each with its own copy of the field memories. One iteration then takes 2048 cycles instead of 8192 — four times faster, for an identical result. The cost falls on memory rather than logic: each lane needs its own memories and read ports, and neighbouring lanes exchange a single

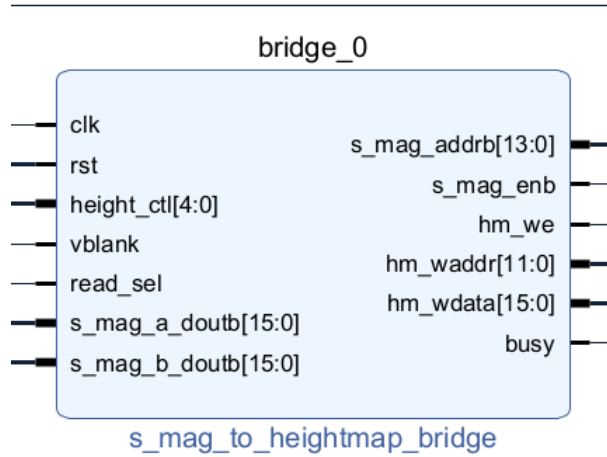


Figure 5.7: The `s_mag_to_heightmap_bridge`: it reads the front buffer (`read_sel`, `s_mag*_doutb`), waits for `vblank`, and writes the renderer heightmap (`hm_we/waddr/wdata`).

boundary row, because the update at the edge of one lane reaches into the next. Notably the four-lane version used *fewer* block RAMs than the single-lane one (108 against 114), because four smaller memories pack into the device’s RAM blocks more tightly than one large one. We also evaluated eight and sixteen lanes but settled on four: beyond that point the additional speed no longer justifies the growing boundary-exchange logic and routing congestion, particularly as the solver was already running well ahead of the display’s frame rate.

5.10 The Two Demonstrations

The same solver drives two different renderers, and the choice between them is primarily a trade between block-RAM use and clock speed (Table 5.3). The first is a flat top-down contour heatmap (Fig. 5.10). It needs only a single heightmap memory, and freeing the rest of the block RAM is what allowed us to quadruple the grid to 128×128 for finer detail; with the source set moving it clearly shows the wavefronts bunching ahead and stretching behind, the Doppler effect. The second renders the field as a shaded three-dimensional landscape by ray-marching (Fig. 5.11). This view is more detailed but more resource-intensive — it needs around fifty copies of the heightmap — so it runs at 100 MHz and folds its memory reads across several cycles to fit, and its camera can be orbited live from the notebook. Keeping both is deliberate: the 2D view is the clearer one for reading the physics, and the 3D view gives a more intuitive sense of the field’s shape.

	2D contour	3D ray-march
Heightmap BRAMs	1	~50
Clock	25 MHz	100 MHz (1 px / 4 cyc)
Strength	clarity, finer grid	visual impact, movable camera
Interaction	moving source (Doppler)	orbiting camera

Table 5.3: The two demonstrations and their block-RAM versus clock trade-off.

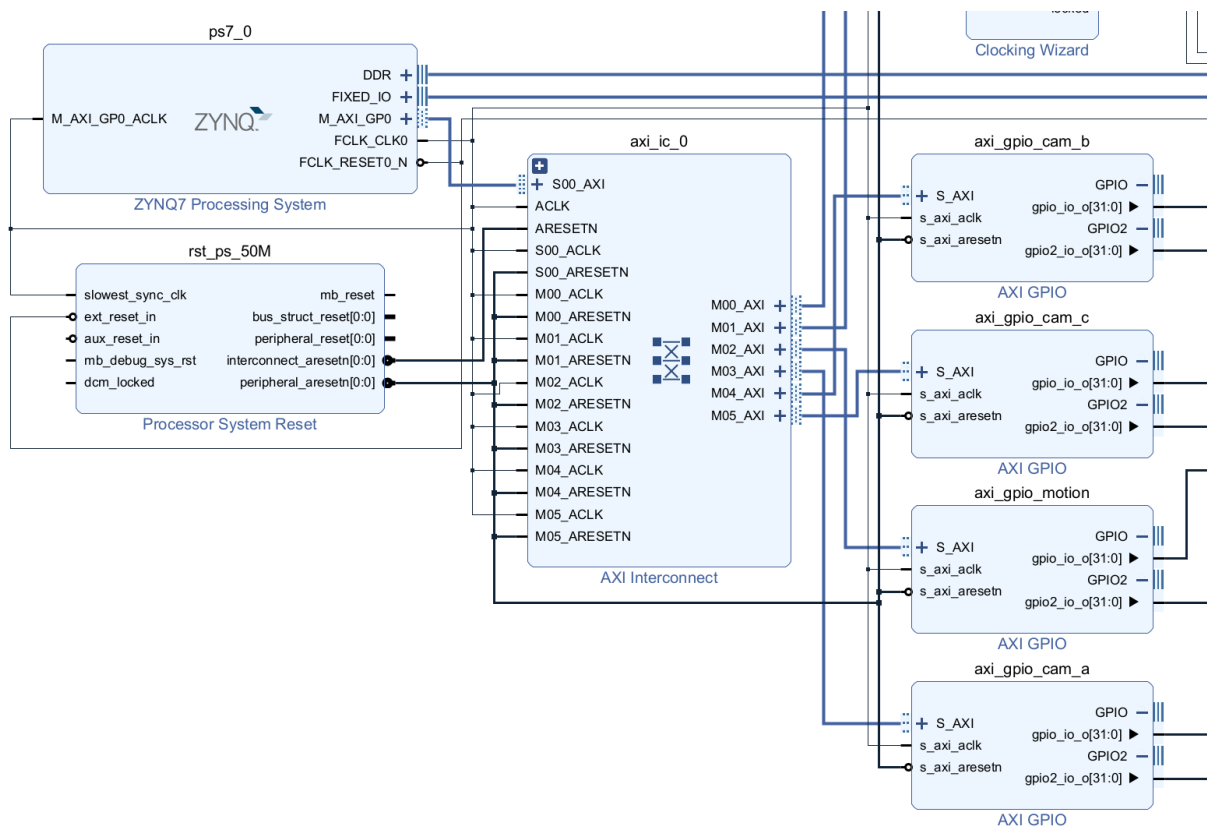


Figure 5.8: The processor (ZYNQ PS7) and its reset controller driving the AXI interconnect, which fans out to the GPIO register blocks (control, motion, status, and the camera channels).

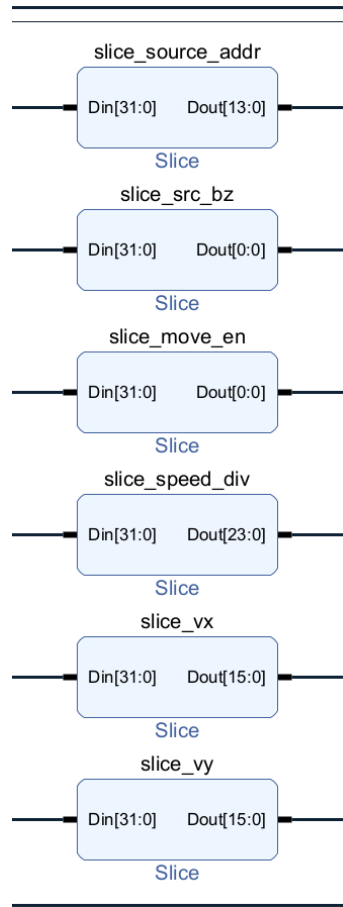


Figure 5.9: Slice blocks unpacking a single 32-bit GPIO register into separate control fields — here the source address, source-field select, and the moving -source velocity and throttle.

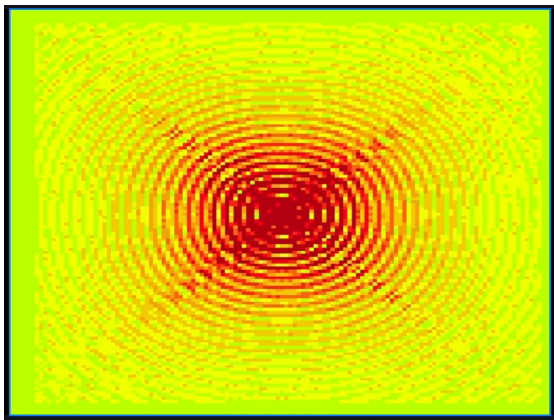


Figure 5.10: 2D contour heatmap: concentric wavefronts radiating from the point source.

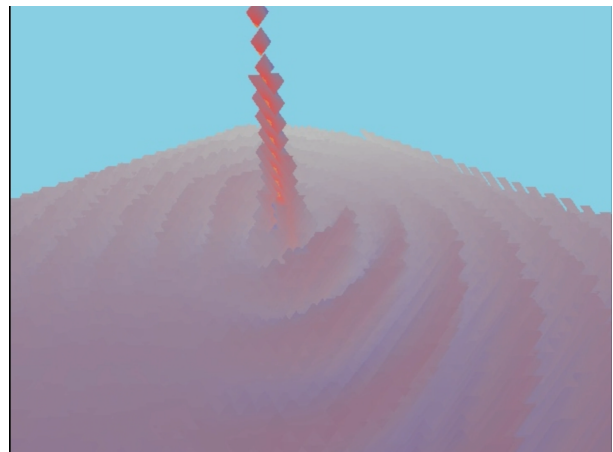


Figure 5.11: 3D ray-marched view: the field as a shaded heightmap, with the source rising at the centre.

Chapter 6

User Interface, Hardware and Demonstrations

Image placeholder — photograph of the finished demonstrator in use: the console with the touch panel, the front row of knobs and switches, and the FPGA board visible in its open bay, connected to a monitor displaying the field
replace with `\includegraphics`

Figure 6.1: The finished demonstrator in use. (*Insert photograph.*)

6.1 What the demonstrator does, and the physics behind it

Electromagnetic fields are invisible and abstract, and the equations rarely make them intuitive. The demonstrator makes them tangible, letting a person interact with the simulated field and watch it respond in real time (Table 6.1).

Property	Interactive Waves
Physics shown	Time-dependent wave (Maxwell)
Input device	Stylus on the touch panel
Panel reports	Stylus XY position
Live knob	Conductivity sets the wave speed

Table 6.1: The interactive wave demonstrator.

Interactive waves. The same surface becomes a canvas: a stylus plants a source, drops a wall, or erases, and the full time-dependent solver shows the wave propagate, reflect, interfere and diffract. A knob scales the propagation speed, so the medium can be slowed, sped up, or made lossy live.

The key hardware requirement is an accurate (x, y) position from the touch surface. This keeps the final design simple and lets the user place sources, walls, or erase directly on the simulation grid. The demonstrator is one block in the wider system: the microcontroller gathers every control and sends a fixed message by wire to the FPGA board, which forwards it to the renderer driving the monitor (Figure 6.3).

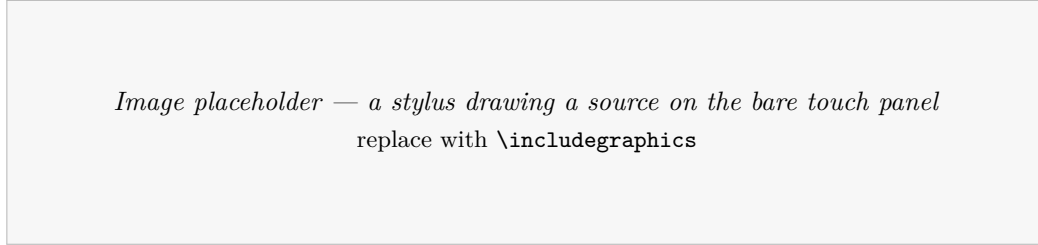


Figure 6.2: Using the stylus to place a source on the touch panel. (*Insert photograph.*)

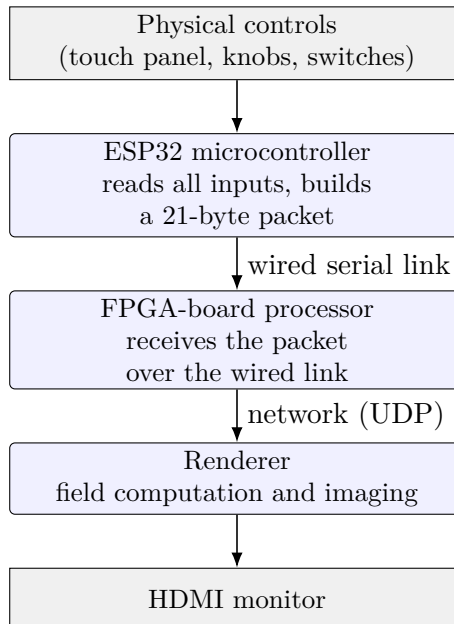


Figure 6.3: Data path of the demonstrator within the wider system. This chapter covers the top two blocks; everything below the serial link belongs to the other teams.

6.2 The user interface

All controls sit on one front face (Table 6.2): one touch surface, four switches and seven rotary knobs. The field-type control is a knob read as three zones, because no three-position switch was available.

Control	Type	Function
Touch panel	4-wire resistive	XY position
2D/3D	switch	flat or rendered-surface display
Wall/Source	switch	what a stylus press places
Clear	switch	wipe the scene; set at power-up to recalibrate
Amplitude	knob	source amplitude
Conductivity	knob	wave speed
Field type	knob (3 zones)	show E , B or Poynting
Yaw, Pitch, Zoom, Z-scale	4 knobs	3D camera view

Table 6.2: The front-panel controls.

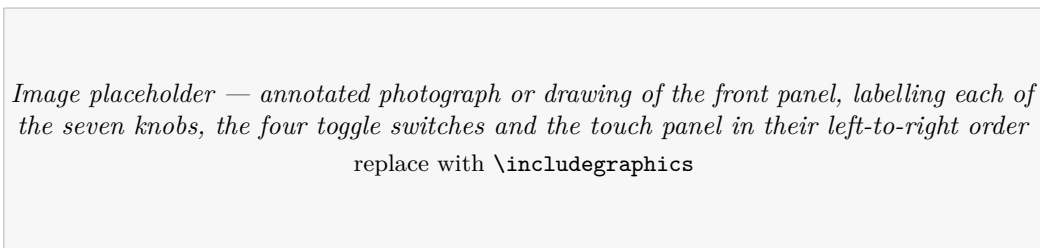


Figure 6.4: The front-panel control layout. (Insert annotated image.)

6.3 Power supply and analog conditioning

The whole subsystem runs from the single 5 V USB feed it shares with the FPGA board, which also ties their grounds together. The microcontroller board regulates this supply to 3.3 V for the controls. The rails are listed in Table 6.3.

The seven knobs are referenced directly to the board’s regulated **3.3 V (3V3) pin**: each is a potentiometer between 3V3 and ground feeding an analog input, so full rotation spans the converter’s full range. The pots together draw only a few milliamps, so the pin holds its voltage and no buffered reference rail is needed. The switches simply pull their pin to ground, held high otherwise by internal pull-ups.

The analog controls require no additional conditioning stage: each potentiometer is connected directly between 3.3 V and ground, with its wiper feeding an ESP32 analog input (Figure 6.5).

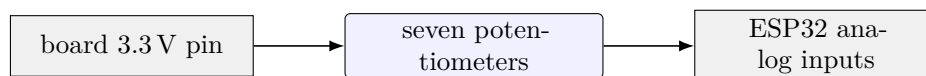


Figure 6.5: Analog-control path: the seven potentiometers are referenced directly to the board’s 3.3 V pin and read by the ESP32 analog inputs.

Each sub-circuit was verified in SPICE before assembly.

Rail	Where it comes from	What it feeds
+5 V	USB supply (shared with the FPGA)	microcontroller board and shared system supply
3.3 V	the microcontroller’s on-board 3V3 pin	the top end of all seven knobs
Ground	common to everything	microcontroller, touch panel, controls, and FPGA board

Table 6.3: The supply rails of the control subsystem.

6.4 The microcontroller and the control link

The subsystem is built around an **ESP32**: a cheap module with a multi-channel analog-to-digital converter, ample I/O, a hardware serial port and Arduino tooling. It does no physics — it only reads the controls and reports them. Its radio stays off (enabling it would disable a bank of the analog inputs in use), and no analog-output (DAC) pins are used — the panel is driven with plain digital levels, and calibration absorbs the small offset.

Eighteen of the roughly two dozen usable pins are occupied (Table 6.4). This effectively fills the chip: a couple of controls sit on an input-only pin and on two start-up (“strapping”) pins, all checked to be harmless, so the board is at the practical limit of a single chip — a fact that drove the design evolution in Section 6.7.

Pins	Purpose
six pins	touch panel — two axis-drive pairs plus two sense lines (Section 6.5)
seven analog inputs	the seven knobs: amplitude, conductivity, field-type, yaw, pitch, zoom, vertical scale
three digital inputs	2D/3D, wall/source and clear switches
two serial pins	transmit and receive for the link to the FPGA board

Table 6.4: How the occupied pins are allocated.

The firmware repeats a fixed cycle about fifty times a second (Figure 6.6), packing everything into a fixed control packet with a start marker and a checksum (Table 6.5). This packet layout is the stable contract with the FPGA team and did not change across hardware revisions. When no touch is valid the position bytes are zero and a flag tells the receiver to place nothing that frame. The ESP32 cannot send UDP itself (its radio is off), so the packet goes by wire to the FPGA board’s processor, which forwards it onto the network to the renderer (Figure 6.3).

6.5 The touch panel: sensing and calibration

Getting clean, trustworthy coordinates from the panel took more effort than any other part.

How it works. The panel is two resistive layers held a hair apart; a press shorts them at the contact point (Figure 6.7). Position is read by driving one layer as a voltage ramp and sensing the contact voltage on the floating layer, swapping the layers for the two axes — three short phases per loop (Table 6.6).

Clean coordinates. Each axis is sensed on a dedicated low-leakage input, so the high-impedance contact node stays linear, and read with the converter’s millivolt calibration. Readings

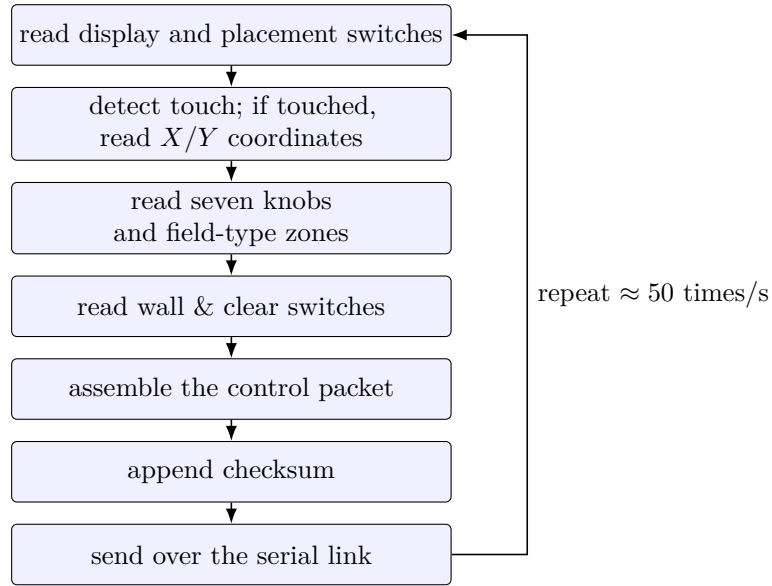


Figure 6.6: The fixed firmware cycle, repeated about fifty times per second.

Bytes	Contents
1	start marker (a fixed recognisable byte)
1	flags: 2D/3D, field type, wall-or-source, clear active, touch valid
2 + 2	touch position, as a column and row on the display grid
2	source amplitude
2	conductivity (wave speed)
2 + 2 + 2 + 2	the four camera-view knobs: yaw, pitch, zoom, vertical scale
1	checksum over the message body

Table 6.5: The fixed control packet.

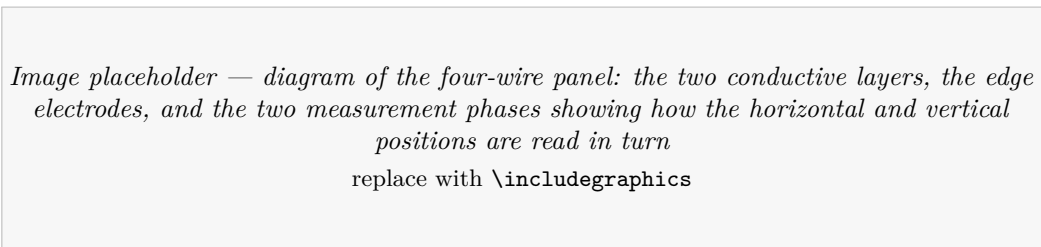


Figure 6.7: Reading a four-wire resistive panel. *(Insert diagram.)*

Phase	Layer driven	Sensed on	Result
Touch detect	Y- pulled low	X+ via pull-up	pressed?
Read X	X layer (X+ high, X- low)	a Y terminal	X position
Read Y	Y layer (Y+ high, Y- low)	an X terminal	Y position

Table 6.6: The three read phases of the four-wire panel.

are filtered — a median for touch detection, a trimmed mean of a dozen samples per coordinate — after a short settling pause. **Orientation** (axis swaps and mirroring) is always corrected in software, never by rewiring.

Calibration. The corners never read the true extremes, so the user sets the range with a quick **two-corner** press — one corner, then the opposite — and only firm presses are learned. The range is saved to non-volatile memory and reloaded at boot, and can be redone by holding the clear switch at power-up. Each press is then converted as in Figure 6.8, and the finished grid cell goes straight into the packet.

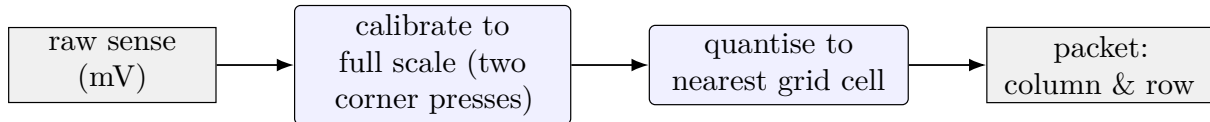


Figure 6.8: Touch-coordinate pipeline: each raw reading is calibrated, quantised to a display-grid cell, and sent in the packet.

6.6 Enclosure and mechanical design (box modelling)

A showcase demonstrator needs a real housing, so a custom enclosure was modelled and 3D-printed, evolving toward the same simplicity as the electronics. It is a single **open-bottomed cover** — an upturned tray — that drops over the baseboard-mounted electronics: no internal fixings, a clean screwless top face, the FPGA left open in its bay, and nothing to trap heat. Its key features are listed in Table 6.7 and its top-face layout in Figure 6.9.

Feature	Size / detail
Overall cover (W×D×H)	318 × 172 × 55 mm
Walls / top plate	3 mm / 5 mm
Touch-panel window	stepped recess + lip, back-left;
FPGA bay	~120 × 90 mm open cut-out, back-right (HDMI/USB/buttons stay exposed)
Control strip	front row of 11 holes — 7 knobs (7 mm dia.) and 4 toggle switches
Cable slot	~40 × 22 mm, back wall
Companion parts	panel frame (bezel)

Table 6.7: Key features of the 3D-printed cover.

The neatest detail is the panel mount: the panel drops into a shallow stepped recess and rests on a thin internal lip, leaving its active area exposed through a window while its bezel sits flush with the top. A slim **panel frame** clamps the touch panel into the recess and keeps it flush with the top face (Figure 6.10).

The model is **parametric**. Every dimension is a named value, so a single edit regenerates a consistent model if a part size changes, and was kept in more than one CAD tool sharing the same values. It is printed **upside-down** so every opening forms with no overhangs and needs no supports (PLA or PETG). At 318 mm it exceeds a typical print bed, so it is split into two halves joined at a seam; the finished enclosure is therefore two box parts plus the panel frame. Boards mount to the baseboard, the framed panel and the knobs and switches fit the top face, the halves join, and the cover drops over with cables exiting the rear slot. Being a simple cover

Image placeholder — the dimensioned top-view layout of the cover, marking the panel window, the FPGA bay, the eleven-hole control strip and the rear cable slot
 replace with `\includegraphics`

Figure 6.9: Top-view layout of the cover. *(Insert dimensioned drawing.)*

Image placeholder — photographs of the printed panel frame, ideally showing how it clamps the touch panel into the enclosure
 replace with `\includegraphics`

Figure 6.10: The printed touch-panel frame. *(Insert photographs.)*

it has no internal standoffs and a single panel window — exactly what the final one-panel design needs.

Image placeholder — a photograph of the printed enclosure, ideally showing the two halves joined and the whole console assembled with the panel, knobs and FPGA in place
 replace with `\includegraphics`

Figure 6.11: The printed and assembled enclosure. *(Insert photograph.)*

6.7 Interactive Wave Simulation

This solves the full time-varying Maxwell equations on the FPGA using FDTD and renders three field quantities simultaneously: the electric field magnitude $|\mathbf{E}|(x, y, t)$, the magnetic field $B_z(x, y, t)$, and the Poynting vector magnitude $|\mathbf{S}|(x, y, t)$. The user places wave sources via the resistive touch panel and can change material properties, and add reflecting boundaries.

6.7.1 Panel 1: Electric field $|\mathbf{E}|(x, y, t)$

Panel 1 shows the scalar magnitude $|\mathbf{E}| = \sqrt{E_x^2 + E_y^2}$. The display shows expanding circular rings centred on the source position.

With two sources, the fields superimpose.

In 3D mode, $|\mathbf{E}|$ is mapped to height. Wavefronts appear as sharp propagating ridges. At constructive interference positions the ridges are tall; at destructive positions they are absent. The ridges travel outward at speed v .

6.7.2 Panel 2: Magnetic field $B_z(x, y, t)$

Panel 2 shows $B_z(x, y, t)$ with the same bipolar colour map as Panel 1. It also shows expanding rings, but they are spatially and temporally shifted relative to those in Panel 1.

The Faraday equation couples $\mathbf{E}$ and $\mathbf{B}$: $\partial B_z / \partial t \propto \nabla \times \mathbf{E}$. So B_z is proportional to the time integral of the curl of $\mathbf{E}$. Integrating a sinusoidal function shifts the phase by 90° :

$$E(t) = E_0 \sin(\omega t) \Rightarrow B_z(t) \propto \int E dt \propto -\cos(\omega t) = \sin\left(\omega t - \frac{\pi}{2}\right) \quad (6.1)$$

This is visible on the display, as when the electric field rings are at maximum, B_z is minimum), and vice versa. The two fields take turns, alternating stored energy at twice the source frequency.

This alternation is the mechanism sustaining wave propagation. The changing $\mathbf{E}$ continuously generates $\mathbf{B}$ via Ampère–Maxwell (3.2), and the changing $\mathbf{B}$ continuously regenerates $\mathbf{E}$ via Faraday. Neither field alone is sufficient, the wave requires both to self generate.

6.7.3 Panel 3: Poynting vector $|\mathbf{S}|(x, y, t)$

Panel 3 shows $|\mathbf{S}|$ computed from the FDTD arrays using

$$\mathbf{S} = \frac{\mathbf{E} \times \mathbf{B}}{\mu_0} \quad (6.2)$$

$|\mathbf{S}| \geq 0$ always. From a single source the display shows a bright central region fading radially outward. With two sources it shows bright fringes where energy concentrates and exactly black nulls where it does not.

The Poynting vector is:

$$\mathbf{S} = \frac{\mathbf{E} \times \mathbf{B}}{\mu_0} \quad (6.3)$$

Both $\mathbf{E}$ and $\mathbf{B}$ are zero at destructive interference nulls. Therefore $|\mathbf{S}| = 0$ exactly at those positions.

The time-averaged Poynting vector is:

$$\langle \mathbf{S} \rangle = \frac{1}{2\mu_0} \text{Re}(\mathbf{E} \times \mathbf{B}^*) \quad (6.4)$$

In the far field, $\langle \mathbf{S} \rangle$ points radially outward, meaning energy radiates away permanently. Near the source ($r \lesssim \lambda/2\pi$), the instantaneous Poynting vector oscillates back and forth without escaping, this is the reactive near field.

6.7.4 Relationship between the three panels

The three panels together give a complete physical picture:

- Panel 1 ($|\mathbf{E}|$): where the electric force on charges is strong.
- Panel 2 (B_z): the magnetic field, 90° out of phase with Panel 1, generated by the changing electric field via Faraday’s law.
- Panel 3 ($|\mathbf{S}|$): where electromagnetic energy flows and at what rate. This is why these 3 are the main ones.

6.7.5 More demonstrations

Reflection and diffraction

Setting $E_z = 0$ at a row of grid cells imposes a perfect electric conductor boundary condition, the tangential electric field must vanish at a conductor surface. Incident waves reflect with inverted phase, and so the superposition of incident and reflected waves forms a standing wave in front of the wall. A gap allows transmission, with waves spreading out in a semicircle on the far side. The angular spread scales as λ/d , where d is the gap width.

Refraction at a dielectric interface

Different ε_r values in the left and right halves of the grid give different wave speeds via (3.5). Wavefronts must remain continuous across the boundary, so they bend: Snell's law $n_1 \sin \theta_1 = n_2 \sin \theta_2$ (with $n = \sqrt{\varepsilon_r \mu_r}$) describes the bending angle. Adjusting the ε_r slider changes the refractive index ratio and the bend angle updates in real time.

Energy density

The electromagnetic energy density:

$$u = \frac{\varepsilon_0}{2}(E_x^2 + E_y^2) + \frac{B_z^2}{2\mu_0} \quad (6.5)$$

is always non-negative. In normalised simulation units this simplifies to $u = E_x^2 + E_y^2 + B_z^2$. At destructive interference nulls both $\mathbf{E}$ and $\mathbf{B}$ vanish, so $u = 0$ exactly. The energy density display complements Panel 3 (poynting vector) $|\mathbf{S}|$, which shows where energy flows, while u shows where energy is stored.

Chapter 7

Evaluation and Conclusion

7.1 Integrated system performance

7.2 Requirements met

7.3 Limitations and future work

7.4 Conclusion